GrappStar build guide

One guide from parts to first mission: print, wire, flash, calibrate, test. Every hands-on step is in order. Every wiring step starts with the battery unplugged.

Built 2026-11-03 11:15 Sydney time from commit `a2e4693`.

How to use it: read section 1 and 2 first, then work through the "Order of work" list in section 1. Where a section says "From `some/file.md`", that text comes straight from the repo, so it is always the latest version. Re-build the guide after any change: `python3 docs/build/make_guide.py`.

1. What the system is 3

Order of work . . . 3

2. Safety 4

Electrical . . . 4

Mechanical . . . 4

3. Parts 5

The only purchase: 608 bearings . . . 5

Base station and rover . . . 5

Dart (KEDGE v30) . . . 5

Tools . . . 5

4. Printing 6

Dart prints . . . 6

Base prints . . . 6

5. Base assembly 7

6. Dart build 8

What changed from the run sheet . . . 8

Dart build order (follow this; it merges the three documents) . . . 8

Dart wiring (new pin map) . . . 8

Dart bench tests . . . 13

7. Electronics and wiring (base and rover) 15

7.1 The whole system . . . 15

7.2 Base ESP32 wiring (pin by pin) . . . 19

8. Flashing both ESP32s 20

8.1 Base: ESP32-S3 DevKitC-1-N8 . . . 20

8.2 Dart: XIAO ESP32S3 . . . 20

9. Jetson setup 21

GrappStar rover software (Jetson) . . . 21

10. Calibration (base) 28

11. Bench tests 29

11.1 Base . . . 29

11.2 Dart . . . 29

12. First tests, bench to field 30

First tests: bench to field . . . 30

Expected numbers (physics model) . . . 32

13. Troubleshooting 34

Base . . . 34
Rover and video . . . 34
Dart . . . 34
14. Measure list 35
What to measure or confirm . . . 35
Numbers still assumed in params.py right now . . . 38
15. Sources and appendices 39

1. What the system is

GrappStar is a rover that can pull itself out of trouble. A launcher on the rover fires a small dart (KEDGE) on a Dyneema tether. The dart lands, screws itself into the soil, and the rover winches against it. Then the dart lets go and is reeled home.

Three parts:

- **Rover.** Jetson Orin Nano Super on the rover's 3S LiPo. It runs the mission, the operator web page, and the vision (rover camera Razer Kiyo X, and the dart's video through an RC832 receiver and a DIGITNOW USB capture).
- **Base station** (on the rover). ONE 12 V wiper motor drives everything. A sliding dog collar on the main shaft couples the wiper to the LAUNCH train, to nothing (NEUTRAL), or to the REEL train. A small N20 motor turns a selector drum (like a motorcycle gear drum) that moves the collar and also lays and removes the capstan wraps.
  - LAUNCH, wiper forward: draws the spring carriage back, fires at the end of the stroke, resets.
  - LAUNCH, wiper backward: tilts the rail up and down to aim. A self-locking worm holds the angle.
  - REEL, wiper forward: the capstan winds in; the storage reel takes up the slack through a slip clutch.
  - Payout at launch: the line skips the capstan and pulls off the free-spinning storage reel against a light payout brake (set 0.5 to 0.75 N; over 1 N costs range).
  - Any fault: wiper off, drum to NEUTRAL. There is a hand lever on the drum shaft end.
  - Brain: an ESP32-S3 DevKitC on USB from the Jetson, with an AS5600 angle sensor on the main shaft and printed contact switches (an M3 screw head touching an M3 nut).
- **Dart (KEDGE v30).** 42 mm x 152 mm, its own XIAO ESP32S3, 2S 18350 pack, WiFi access point. After landing a kick leg swings over the nose and plants a foot auger, then two drill pods swing down and screw in. A nose camera sends analog video at 5.8 GHz.

How they talk: the Jetson talks to the base over USB serial (JSON lines, `firmware/base/PROTOCOL.md`) and to the dart over WiFi (HTTP at 192.168.4.1, `firmware/dart/API.md`). Video is analog 5.8 GHz into the USB capture.

A mission: pick a target on the rover camera view, the Jetson looks up the pitch in the ballistics table, the base aims, cocks and fires on your confirmation, the dart video is analysed for the landing zone, the dart anchors, the base wraps the capstan and tows, the dart releases, the base reels it home.

Order of work

1. Measure the wiper motor shaft, mount and housing (section 14, A1 to A5) and the springs (group B). Put the numbers in `base/spec/params.py` and re-run the base pipeline. The base parts that touch the motor depend on them. A6 and A7 (speed, current, torque) need the base electronics: do them at test stage 1. They change no printed part.
2. Confirm the parts box (section 14, group C). The only purchase is 608 bearings.
3. Print the dart fit coupon, then the dart plates (section 4). The dart does not depend on the measurements.
4. Flash the dart and build it (sections 6 and 8, and the dart run sheet in the appendix).
5. Print the base (section 4) and assemble it (section 5).
6. Set up the Jetson on its own wall supply (section 9). Flash the base ESP32 from the Mac or the Jetson, with nothing else connected to it (section 8.1).
7. Wire the base and the rover (section 7) and do the first power-on sequence (7.1, section 4). While the Jetson is still on its wall supply, leave the barrel plug out in step 7 and plug the ESP32 into the Jetson in step 8.
8. Calibrate (section 10) and bench test (section 11).
9. Run the staged tests, bench to field (section 12).

2. Safety

Read this before any hands-on step.

Electrical

- **Battery unplugged before you touch, add or move any wire.** On the dart that means the power loop plug out. The first XIAO died from wiring with the battery live.
- Battery unplugged before you plug any board into the Mac by USB.
- Check for shorts with the meter on beep before the first power-on of anything you wired: red to black must not beep.
- The 3S LiPo can deliver over 100 A into a short. Keep the 20 A and 10 A fuses in their leads. Never bypass them.
- LiPo care: charge on the iMax B6 in balance mode, on a non-flammable surface, never unattended. Stop using the pack at 3.3 V per cell under load (9.9 V). A puffed pack goes in a LiPo bag and to battery recycling.
- 18350 cells: never let both cells touch metal at the same time. Charge out of the dart in the XTAR, or through the tail balance socket with the loop plug out.
- Never connect the DRV8833 motor supply (VM) to the LiPo directly: 10.8 V maximum. It runs from the 9.0 V buck.
- The dart VTX must have its antenna on before it is powered.

Mechanical

- The wiper motor is strong (about 15 N m) and self-locking. Hands out of the gears, drum and rail while the battery is plugged in. Pull the battery plug to stop everything.
- The launch springs store energy. Cocked carriage = loaded. Never put a hand in front of the rail or the carriage. To unload a cocked base without firing, see troubleshooting.
- From the first spring test on: safety glasses, nobody in front of the rail, the area clear for 30 m.
- Augers and drills are sharp and turn with force. Hold the dart by the hull, never near a pod or the foot.
- Soldering: iron on its stand, fan on for fumes, 3 s at most per joint near PLA. Heat gun on low, no lighter near PLA.

3. Parts

Everything below is already in hand except the 608 bearings. Section 14 group C lists the items to confirm by looking in the box.

The only purchase: 608 bearings

- Search: "608ZZ bearing 8x22x7" (or "608-2RS", sealed). Buy 8 to 10. Skateboard bearings are 608s and fine.
- Specs: 8 mm bore, 22 mm outside, 7 mm wide, rated well over 10,000 rpm.
- Uses: storage reel (2), capstan or wrap arm (2), rollers and spares. Axles: M8 bolt or 8 mm rod (check the box).

Base station and rover

Part	Qty	Note
12 V wiper motor, self-locking worm	1	Rebuilt to 2 wires, park linkage removed. Measure it first (section 14 A)
BTS7960 motor driver module	1	Drives the wiper
ESP32-S3 DevKitC-1-N8	1	Base brain, USB to the Jetson
Adafruit AS5600 (STEMMA QT) + diametric magnet	1	Main shaft angle
DRV8833 carrier	1	DRV8833 #3, freed from the dart by the new dart pin map
MP1584 buck	1	MP1584 #2 from the dart build, set to 9.0 V
N20 right-angle worm motor	1	Selector drum (count your N20s: section 14 C3)
CSK20PP one-way bearings (20 x 47 x 14)	2	#1 spring cock and fire, #2 capstan backstop
Century 16 x 165.1 x 1.4 mm extension springs	2	Launch springs (Bunnings 0487961). Measure them (section 14 B)
608 bearings	8 to 10	The purchase
Dyneema braid 150 lb, 1 mm	20 m	Tether
100 kg load cell + HX711	1	Optional line tension
3S LiPo 5200 mAh (Zeee), iMax B6 charger	1	Rover battery
XT60 lead #1 (10 A fuse), lead #2 (20 A fuse), spare XT60s, fuse holder		Lead #2 feeds the wiper; build the Y harness from spares
Barrel plug lead 5.5 x 2.5 mm	1	Jetson from the LiPo
Jetson Orin Nano Super	1	
Lockmaner RC832 receiver, DIGITNOW USB capture, RCA lead	1 each	Dart video in
Razer Kiyo X	1	Rover camera
Resistors 10 k x 2, 1 k, 680 R, 100 k, 22 k		Pull-downs and dividers (firmware/base/WIRING.md)
M3 screws, M3 nuts, M3 brass heat-set inserts		Frame joints and printed contacts
Dupont jumpers, 14 / 18 / 22 / 26 AWG wire, heat shrink		

Dart (KEDGE v30)

The full dart list with quantities is in the dart run sheet, section 2 (appendix). In short: XIAO ESP32S3, 2 x DRV8833 (the third goes to the base), INA219, BNO085, MP1584 #1 (5.04 V), OTOTEC 2S BMS, 2 x 18350 cells, 3 worm N20s (kick, deploy, foot), 2 drill gearmotors, 5 x F693ZZ, Caddx Ant, Zeus VTX and antenna, JST-XH sockets, Schottky diode, capacitor, M3 hardware, 15 g nose ballast.

Tools

Soldering iron and solder, flux, multimeter, wire strippers, flush cutters, small flat screwdriver (MP1584 trimmer), heat shrink and heat gun, caliper, hex keys, tweezers, kitchen scale, luggage scale, tape measure, safety glasses, 5-minute epoxy, threadlocker, zip ties, foam tape.

4. Printing

Printer: Bambu X2D (bed 256 x 256 x 260 mm). PLA for everything except loaded gears and keyed shafts, which are PPA-CF (the print list says which).

Order: dart fit coupon first (20 minutes, checks your printer's hole sizes), then the dart plates, then the base once the wiper and spring numbers are in (section 1, order of work).

Dart prints

Dart plates (in `kedge/pack_v30/`, also inside `kedge/dist_v30/KEDGE_v30_build_pack.zip` under `print/`):

- `kedge/pack_v30/plate_A_hull.3mf`
- `kedge/pack_v30/plate_B_legs_tray.3mf`
- `kedge/pack_v30/fit_coupon.stl` (print this first, 20 minutes)

Dart part	Print pose	Size on bed mm	Volume cm3
hull_top	split face down (on the bed), saddle up	121 x 50 x 35	17.41
hull_bottom	split face down, belly up	121 x 42 x 21	11.5
nose	tip down on the bed (flat Ø22), spigot up	42 x 42 x 27	8.25
tail_cap	end face down, spigot up (no supports)	42 x 42 x 15	7.13
tray	keel down on the bed; discs stand up, decks bridge 26 mm between discs	109 x 39 x 36	15.98
kick_leg	on its side (pole axis horizontal, hub face down)	119 x 15 x 15	8.7
drill_leg_L	upright, motor pocket open to the top (as installed folded)	46 x 15 x 13	3.37
drill_leg_R	upright, motor pocket open to the top (as installed folded)	46 x 15 x 13	3.37
drill_leg_L_auger	base down, nut slot sideways	17 x 17 x 73	5.5
drill_leg_R_auger	base down, nut slot sideways	17 x 17 x 73	5.5
foot_auger	base down	17 x 17 x 40	3.06
coupler_L	standing on its socket end	7 x 6 x 36	1.24
coupler_R	standing on its socket end	7 x 6 x 27	0.9

Settings: PLA, 0.4 nozzle, 0.16 mm layers, 4 walls, 40% gyroid infill, 5 top and bottom layers, supports off, brim on for the augers and couplers. Hole compensation from the fit coupon (run sheet section 4).

Base prints

The base print pack comes from the base pipeline (`base/pack`). It holds every part already in print orientation, the plates as 3MF, and the material for each part.

> *NOT IN THE REPO YET:* `base/pack` *(written by cad-infra (print pack)). Re-run* `python3 docs/build/make_guide.py` *after it lands and this section fills in.*

Before printing the base:

1. The wiper and spring numbers are in params.py and the pipeline was re-run (the audit said CLEAN).
2. Print one test part from the pack in PPA-CF and check a 608 drops into its seat with a firm thumb press, and an M3 heat-set insert sets straight. Adjust hole compensation the same way as the dart coupon.
3. PPA-CF: dry the spool first (it absorbs water), hardened nozzle, settings from the Bambu PPA-CF profile.

5. Base assembly

The assembly steps come from the base pipeline: every step is generated from the same model that passed the collision audit, with a picture per step. They are in `base/guide` (PDF, also attached as an appendix) and `base/out`.

> *NOT IN THE REPO YET:* `base/guide/README.md` *(written by cad-infra (assembly steps)). Re-run* `python3 docs/build/make_guide.py` *after it lands and this section fills in.*

General rules for the base:

- Heat-set inserts: soldering iron at about 220 C with the insert tip, press straight in, let it cool before loading.
- Bearings: press in square with a flat block, never hammer on the inner race.
- Keyed shafts: check the key slides in the sprag (CSK20PP) keyway by hand before final assembly.
- Turn every train by hand (battery unplugged) after each step: it must turn without binding before the next step.
- Leave the springs off until test stage 4 (section 12).

6. Dart build

The dart is built from three documents. This section says which to use for what, and in what order.

- **KEDGE v30 run sheet** (appendix at the end, also `kedge/pack_v30/KEDGE_v30_run_sheet.pdf`): printing, board prep, mechanical assembly, function test. Its own section numbers are used below.
- `firmware/dart/WIRING.md` (below): the new wiring. It replaces the run sheet's wiring (its sections 7 and 9 and its wiring picture).
- `firmware/dart/BENCH_TEST.md` (below): the bench checks for the new firmware.

What changed from the run sheet

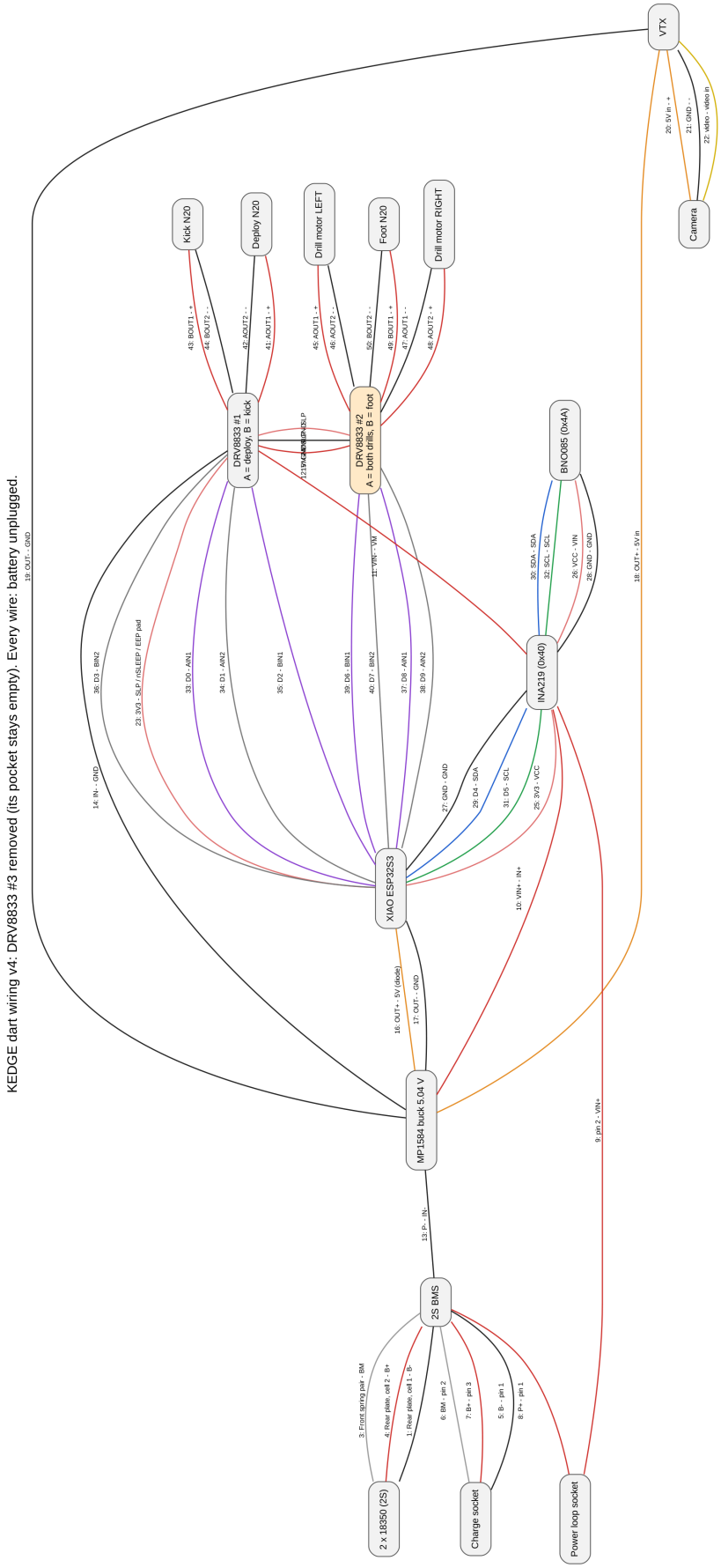
- The dart now uses two DRV8833 boards. DRV#3 goes to the base selector drum; its pocket on the keel stays empty.
- DRV#2 channel A drives BOTH drill motors (right drill leads still swapped), channel B drives the foot.
- No pin bridge on DRV#2 (ignore the bridge in run sheet 7.3).
- To reverse a motor, swap its two thin input wires at the driver instead of the motor leads (run sheet 7.6 and 10 swap motor leads by v29 number): drills = D8 / D9 at DRV#2 AIN1 / AIN2, foot = D6 / D7 at DRV#2 BIN1 / BIN2, deploy = D0 / D1 at DRV#1 AIN1 / AIN2, kick = D2 / D3 at DRV#1 BIN1 / BIN2. Exception: if only ONE drill turns the wrong way, swap that drill's motor leads.
- **Wire numbers changed.** The tape flags carry the NEW numbers from the new cut list. Wherever the run sheet names a wire number (sections 7.6, 8b, 8d, 8e, 8f and 10), look it up in the table "Run sheet wire numbers" below.
- Use the firmware in `firmware/dart/`, not the copy inside the v30 pack. Ignore the run sheet's `firmware/reel` sketch: `firmware/base/` replaces it. The Adafruit INA219 library is no longer needed (BNO08x still is).
- Run sheet section 10, "Put them in kedge_logic.h (kickContactMa, kickStallMa, deployStallMa)": those names are gone. Enter the measured currents as BENCH_TEST.md says.
- Foot direction: the foot auger is right-hand thread, so it screws in when it turns **clockwise seen from the motor end, looking toward the auger tip**. The run sheet's "clockwise, seen from the shaft end" in 7.6 means the same only if you stand behind the motor; use the motor-end wording.

Dart build order (follow this; it merges the three documents)

1. Run sheet section 4: print and check the fit coupon.
2. Run sheet section 5: start plate A, flash the firmware from `firmware/dart/` (section 8.2 of this guide).
3. Run sheet section 6 steps 1 to 3: clip headers, set the buck. Set MP1584 #1 to 5.04 V and MP1584 #2 to 8.0 V from the 3S LiPo on XT60 lead #1 (run sheet 6.2), before either buck is soldered to anything. (This replaces WIRING.md step B2.) MP1584 #2 later becomes the base drum supply at 9.0 V.
4. Run sheet section 6 step 4: find the SLP pad on DRV#1 and DRV#2 (WIRING.md step B6). Meter on resistance: SLP to VM must read open.
5. Battery unplugged. Solder the bench harness with WIRING.md Path B, steps B1 and B3 to B10, on the bench, outside the hull, motor leads left free at their far ends. Leave the BMS end of the battery wires for step 9.
6. Battery unplugged. **Click the u.FL antenna onto the VTX** (never power the VTX without it). Bench power-up: the 8.0 V MP1584 #2 on the LiPo feeds the INA219 VIN+ pad and main ground, exactly as run sheet 7.5. Check the readings there (about 4.7 V at the XIAO 5V pad, VTX LED on, z finds 0x40 and 0x4a).
7. Motor directions on the bench (BENCH_TEST.md, the steps that need only a loose motor; run sheet 7.6 with the new wire numbers). Mark each motor lead. Disconnect the bench supply.
8. Start plate B. Run sheet 8a: nose ballast.
9. Battery unplugged. Run sheet 8b: battery contacts and BMS, using the new wire numbers.
10. Run sheet 8c, 8d, 8e, 8f in order (pods, tray, close the hull, kick leg and foot). The run sheet closes the hull with the kick and drill leads threaded out through their holes, then solders those motors outside: that is intended. Antenna still on before every loop-plug-in.
11. BENCH_TEST.md for everything that needs the legs fitted, then run sheet section 10 (function test in damp soil). Steps that use the Jetson (`kedge_client.py`, the RC832) wait until sections 7 and 9 are done: from the Jetson use `python3 -m grappstar dart status`.

Dart wiring (new pin map)

NOT IN THE REPO YET: `firmware/dart/WIRING.md` *(written by fw-dart). Re-run* `python3 docs/build/make_guide.py` *after it lands and this section fills in.*



Dart wiring v4 (numbers match the new cut list). To zoom in, open firmware/dart/wiring/wiring.svg on a screen.

Run sheet wire numbers

When the run sheet names a wire number, use the new number from this table.

Run sheet wire	New wire	Run sheet says
1	1	Rear plate, cell 1 (-) to BMS B-
2	2	Front spring, cell 1 (+) to Front spring, cell 2 (-)
3	3	Front spring pair (cell 1 + / cell 2 -) to BMS BM
4	4	Rear plate, cell 2 (+) to BMS B+
5	5	BMS B- to Charge socket pin 1 (tail window)
6	6	BMS BM to Charge socket pin 2
7	7	BMS B+ to Charge socket pin 3
8	8	BMS P+ to Power loop socket pin 1 (tail window)
9	9	Power loop socket pin 2 to INA219 VIN+
10	10	INA219 VIN+ (same pad) to MP1584 IN+
11	11	INA219 VIN- to DRV#1 VM
12	12	DRV#1 VM to DRV#2 VM
13	REMOVED (no DRV#3)	DRV#2 VM to DRV#3 VM
14	13	BMS P- to MP1584 IN-
15	14	MP1584 IN- (same pad) to DRV#1 GND
16	15	DRV#1 GND to DRV#2 GND
17	REMOVED (no DRV#3)	DRV#2 GND to DRV#3 GND
18	16	MP1584 OUT+ (5.04 V) to XIAO 5V via a Schottky diode (stripe toward the XIAO)
19	17	MP1584 OUT- to XIAO GND
20	18	MP1584 OUT+ to VTX 5V in
21	19	MP1584 OUT- to VTX GND
22	20	VTX 5V in (same pad) to Camera + (red)
23	21	VTX GND (same pad) to Camera - (black)
24	22	Camera video (yellow) to VTX video in
25	23	XIAO 3V3 to DRV#1 SLP / nSLEEP / EEP pad (NOT VCC or VIN)
26	24	DRV#1 SLP to DRV#2 SLP
27	REMOVED (no DRV#3)	DRV#2 SLP to DRV#3 SLP
28	25	XIAO 3V3 to INA219 VCC
29	26	INA219 VCC to BNO085 VIN
30	27	XIAO GND to INA219 GND
31	28	INA219 GND to BNO085 GND
32	29	XIAO D4 to INA219 SDA
33	30	INA219 SDA to BNO085 SDA
34	31	XIAO D5 to INA219 SCL
35	32	INA219 SCL to BNO085 SCL
36	33	XIAO D0 to DRV#1 AIN1
37	34	XIAO D1 to DRV#1 AIN2
38	35	XIAO D2 to DRV#1 BIN1
39	36	XIAO D3 to DRV#1 BIN2
40	37	XIAO D8 to DRV#2 AIN1 + BIN1 (bridge the two pads)
41	38	XIAO D9 to DRV#2 AIN2 + BIN2 (bridge the two pads)
42	39	XIAO D6 to DRV#3 AIN1
43	40	XIAO D7 to DRV#3 AIN2
44	41	DRV#1 AOUT1 to Deploy N20 + (red)
45	42	DRV#1 AOUT2 to Deploy N20 - (black)
46	43	DRV#1 BOUT1 to Kick N20 + (red)
47	44	DRV#1 BOUT2 to Kick N20 - (black)
48	45	DRV#2 AOUT1 to Drill motor LEFT + (red)
49	46	DRV#2 AOUT2 to Drill motor LEFT - (black)

Run sheet wire	New wire	Run sheet says
50	47	DRV#2 BOUT1 to Drill motor RIGHT - (black) SWAPPED
51	48	DRV#2 BOUT2 to Drill motor RIGHT + (red) SWAPPED
52	49	DRV#3 AOUT1 to Foot N20 + (red)
53	50	DRV#3 AOUT2 to Foot N20 - (black)

New dart wire cut list (replaces run sheet section 9)

From `firmware/dart/wiring/wire_cut_list.csv`*:*

#	from	to	colour	gauge	cut length mm	note	v29 #
1	Rear plate, cell 1 (-)	BMS B-	black	22 AWG	45	solder to the plate tab before it goes in its recess	1
2	Front spring, cell 1 (+)	Front spring, cell 2 (-)	red	22 AWG	45	the series link: carries the full battery current; glue both springs in the front recess	2
3	Front spring pair (cell 1 + / cell 2 -)	BMS BM	white	26 AWG	55	midpoint sense	3
4	Rear plate, cell 2 (+)	BMS B+	red	22 AWG	45		4
5	BMS B-	Charge socket pin 1 (tail window)	black	26 AWG	80	JST-XH 3-pin, balance charge	5
6	BMS BM	Charge socket pin 2	white	26 AWG	125		6
7	BMS B+	Charge socket pin 3	red	26 AWG	80		7
8	BMS P+	Power loop socket pin 1 (tail window)	red	22 AWG	80	JST-XH 2-pin; the shorting plug is the ON switch	8
9	Power loop socket pin 2	INA219 VIN+	red	22 AWG	95	main + after the loop	9
10	INA219 VIN+ (same pad)	MP1584 IN+	red	22 AWG	140	buck taps before the shunt, so INA219 reads motor current only	10
11	INA219 VIN-	DRV#1 VM	red	22 AWG	50	motor rail 6.0-8.4 V	11
12	DRV#1 VM	DRV#2 VM	red	22 AWG	65	daisy chain ends here (no DRV#3)	12
13	BMS P-	MP1584 IN-	black	22 AWG	130	main ground	14
14	MP1584 IN- (same pad)	DRV#1 GND	black	22 AWG	120		15
15	DRV#1 GND	DRV#2 GND	black	22 AWG	65		16
16	MP1584 OUT+ (5.04 V)	XIAO 5V via a Schottky diode (stripe toward the XIAO)	orange	24 AWG	160	set and check 5.04 V BEFORE connecting; 1N5817 or SS14 in heat shrink	18
17	MP1584 OUT-	XIAO GND	black	24 AWG	160		19
18	MP1584 OUT+	VTX 5V in	orange	26 AWG	95		20
19	MP1584 OUT-	VTX GND	black	26 AWG	95		21
20	VTX 5V in (same pad)	Camera + (red)	orange	28 AWG	105	runs forward along the top through the disc notches	22
21	VTX GND (same pad)	Camera - (black)	black	28 AWG	105		23
22	Camera video (yellow)	VTX video in	yellow	28 AWG	105	twist with the camera ground	24
23	XIAO 3V3	DRV#1 SLP / nSLEEP / EEP pad (NOT VCC or VIN)	red/white	28 AWG	55	wakes the driver; meter check first (WIRING.md step 6)	25

#	from	to	colour	gauge	cut length mm	note	v29 #
24	DRV#1 SLP	DRV#2 SLP	red/white	28 AWG	65		26
25	XIAO 3V3	INA219 VCC	red/white	28 AWG	60		28
26	INA219 VCC	BNO085 VIN	red/white	28 AWG	60		29
27	XIAO GND	INA219 GND	black	28 AWG	60		30
28	INA219 GND	BNO085 GND	black	28 AWG	60		31
29	XIAO D4	INA219 SDA	blue	28 AWG	60	I2C SDA	32
30	INA219 SDA	BNO085 SDA	blue	28 AWG	60		33
31	XIAO D5	INA219 SCL	green	28 AWG	60	I2C SCL	34
32	INA219 SCL	BNO085 SCL	green	28 AWG	60		35
33	XIAO D0	DRV#1 AIN1	purple	28 AWG	55	deploy	36
34	XIAO D1	DRV#1 AIN2	grey	28 AWG	55	deploy	37
35	XIAO D2	DRV#1 BIN1	purple	28 AWG	55	kick	38
36	XIAO D3	DRV#1 BIN2	grey	28 AWG	55	kick	39
37	XIAO D8	DRV#2 AIN1 (only AIN1: NO bridge to BIN1)	purple	28 AWG	45	drills	40
38	XIAO D9	DRV#2 AIN2 (only AIN2: NO bridge to BIN2)	grey	28 AWG	45	drills	41
39	XIAO D6	DRV#2 BIN1	purple	28 AWG	45	foot (slow worm, harmless if it twitches at reset)	42
40	XIAO D7	DRV#2 BIN2	grey	28 AWG	45	foot	43
41	DRV#1 AOUT1	Deploy N20 + (red)	red	26 AWG	165	motor sits in the sled cradle	44
42	DRV#1 AOUT2	Deploy N20 - (black)	black	26 AWG	165		45
43	DRV#1 BOUT1	Kick N20 + (red)	red	26 AWG	150	out through the hole in the saddle floor	46
44	DRV#1 BOUT2	Kick N20 - (black)	black	26 AWG	150		47
45	DRV#2 AOUT1	Drill motor LEFT + (red)	red	26 AWG	195	out the left flank hole, 30 mm loop for the pod swing	48
46	DRV#2 AOUT2	Drill motor LEFT - (black)	black	26 AWG	195		49
47	DRV#2 AOUT1 (same pad as the LEFT red)	Drill motor RIGHT - (black) SWAPPED	black	26 AWG	170	reversed on purpose: the pair counter-rotates	50
48	DRV#2 AOUT2 (same pad as the LEFT black)	Drill motor RIGHT + (red) SWAPPED	red	26 AWG	170		51
49	DRV#2 BOUT1	Foot N20 + (red)	red	26 AWG	330	forward along the top through the disc notches, out the skin hole, 60 mm loop round the hub, through the pole channel	52
50	DRV#2 BOUT2	Foot N20 - (black)	black	26 AWG	330	silicone wire, it flexes every deploy	53

Dart bench tests

NOT IN THE REPO YET: `firmware/dart/BENCH_TEST.md` *(written by fw-dart). Re-run* `python3 docs/build/make_guide.py` *after it lands and this section fills in.*

7. Electronics and wiring (base and rover)

Battery unplugged for every step in this section. Build in this order: the power harness (Y lead, fuses), the base electronics bay, the sensors and contacts, the Jetson and video. Then do the first power-on sequence (7.1 section 4) before anything is connected to a motor.

7.1 The whole system

From `docs/build/SYSTEM_ELECTRICAL.md`*:*

GrappStar system electrical (rover, base and dart)

Diagram: `system_electrical.svg` / `system_electrical.png` (source `system_electrical.dot`). The ESP32 pin table is owned by `firmware/base/WIRING.md` and `firmware/base/grappstar_base/pins.h`; this page shows how the whole rover is powered and connected. If this page and WIRING.md ever disagree on a pin, WIRING.md wins.

1. What is on the rover

Block	Part	Powered from
Battery	Zeee 3S LiPo 5200 mAh, XT60 (9.9 V empty in use, 11.1 V nominal, 12.6 V full)	
Wiper branch	XT60 lead #2 with the 20 A fuse, BTS7960 module, 12 V wiper motor (2 wires, park switch taped off)	LiPo
Drum branch	MP1584 #2 buck set to 9.0 V, DRV8833 #3 (the one freed from the dart), selector drum N20 worm	BTS7960 B+ / B- terminals, after the 20 A fuse
Base brain	ESP32-S3 DevKitC-1-N8	Jetson USB (the port marked USB, not UART)
Sensors on the ESP32	Adafruit AS5600 (STEMMA QT, I2C 0x36), printed contacts (M3 screw head on an M3 nut), optional HX711 + 100 kg load cell, battery divider	ESP32 3V3
Jetson	Orin Nano Super dev kit	XT60 lead #1 (10 A fuse) to a 5.5 x 2.5 mm barrel pigtail
Video in	Lockmaner RC832 5.8 GHz receiver, AV (yellow RCA) out to the DIGITNOW USB capture, which plugs into the Jetson	XT60 lead #1 branch (12 V class input)
Rover camera	Razer Kiyo X	Jetson USB
Dart	KEDGE: XIAO ESP32S3, 2 x DRV8833, INA219, BNO085, Caddx Ant, Zeus VTX	its own 2S 18350 pack. No wire to the rover: the tether is Dyneema and carries no power

2. Power tree

```
3S LiPo (XT60)
  -> XT60 Y harness (one male in, two female out; made from your spare XT60s, 14 AWG)
       -> lead #2, 20 A fuse, 14 AWG -> BTS7960 B+ / B-  -> wiper motor (BTS7960 M+ / M-, 14 AWG)
                                     \-> 2 A inline fuse, 22 AWG from the B+ / B- screws -> MP1584 #2 IN (
  9.0 V out)
                                           -> DRV8833 #3 VM / GND -> drum N20 (AOUT1 / AOUT2)
       -> lead #1, 10 A fuse, 18 AWG -> barrel pigtail -> Jetson DC jack
                                     \-> 1 A inline fuse, 22 AWG -> RC832 power in
Jetson USB -> ESP32-S3 DevKitC (5 V from USB; the board makes its own 3V3)
                ESP32 3V3 -> BTS7960 VCC, AS5600, HX711 (DRV8833 #3 gets only AIN1, AIN2, GND)
Jetson USB -> DIGITNOW capture <- RC832 AV out
Jetson USB -> Kiyo X
```

Why it is laid out this way:

- The DRV8833 motor supply (VM) is 10.8 V maximum. The LiPo is 12.6 V full, so the drum N20 runs from MP1584 #2 set to 9.0 V. MP1584 #2 is the spare from the dart build (it was the 8.0 V bench supply there): reset it to 9.0 V after the dart bench test.
- The buck taps the BTS7960 screw terminals, so both motor branches sit behind the 20 A fuse and the Jetson branch stays separate. A wiper stall that blows the 20 A fuse does not reboot the Jetson.
- The thin taps (MP1584 input, RC832) each get their own small inline fuse (2 A and 1 A) right at the tap. A 22 AWG wire behind only the 20 A or 10 A fuse would burn before that fuse blew if it shorted.
- The ESP32 takes power only from USB. It never connects to the LiPo directly. Its ground meets battery minus through the BTS7960 and DRV8833 GND wires (needed so the logic signals have a reference). There is a second ground path through USB, the Jetson and its barrel lead, so if the BTS7960 B- screw came loose the wiper current would try to return through the thin GND jumper and the USB cable. Keep the B- screw tight (tug test) and never run the wiper with B- disconnected.

- The wiper and the Jetson share the pack. A wiper stall (about 18 A) makes the pack voltage sag; with a low pack that sag can brown out the Jetson. Charge the pack when it reads 3.6 V per cell at rest (10.8 V).

3. Wires and connectors

Run	From	To	Gauge	Connector	Note
P1	LiPo	Y harness	14 AWG	XT60	Make the Y from spare XT60s; solder, heat shrink each joint
P2	Y harness	BTS7960 B+ / B-	14 AWG	XT60 lead #2 (20 A fuse), screw terminals at the BTS7960	Red to B+, black to B-. 10 A blew on wiper inrush before: keep 20 A
P3	BTS7960 M+ / M-	wiper motor	14 AWG	screw terminals; spade or solder at the motor	Red wire (+) and black (-) of the rebuilt 2-wire motor
P4	BTS7960 B+ / B-	MP1584 #2 IN+ / IN-	22 AWG	under the same screws, 2 A inline fuse in the + wire right at the screw	Set 9.0 V with nothing on the output first
P5	MP1584 #2 OUT+ / OUT-	DRV8833 #3 VM / GND	22 AWG	solder	Check 9.0 V on the meter before you connect
P6	DRV8833 #3 AOUT1 / AOUT2	drum N20	26 AWG	solder or JST	Swap the pair if the drum turns the wrong way (see calibration)
P7	Y harness	Jetson DC jack	18 AWG	XT60 lead #1 (10 A fuse), 5.5 x 2.5 mm barrel plug (same as the plug on the Jetson supply), centre positive	Check polarity with the meter before the first plug-in
P8	lead #1 (after the fuse)	RC832 power in	22 AWG	the RC832 power lead, 1 A inline fuse in the + wire at the tap	Read the RC832 label for its input range first (see MEASURE_LIST)
S1	Jetson USB-A	ESP32-S3 USB port (native, marked USB)		USB-C or micro-USB cable to suit the board	Data cable, not charge-only
S2	Jetson USB-A	DIGITNOW capture		its own USB plug	
S3	RC832 AV out	DIGITNOW video in		yellow RCA	Video only; audio not used
S4	Jetson USB-A	Kiyo X		its own USB plug	
L1	ESP32 pins	BTS7960 RPWM, LPWM, R_EN, L_EN, IS, VCC, GND	26 to 28 AWG	Dupont jumpers	Pins in firmware/base/WIRING.md
L2	ESP32 pins	DRV8833 #3 AIN1, AIN2, GND	26 to 28 AWG	Dupont	SLP / nSLEEP is not wired: the board pulls it high, and its inputs are pulled low so the outputs are off until the ESP32 drives them
L3	ESP32 SDA / SCL / 3V3 / GND	AS5600		STEMMA QT to Dupont cable	Magnet 1 to 3 mm from the chip
L4	ESP32 pins	each printed contact	26 AWG twisted pair	Dupont at the ESP32, M3 ring or solder at the contact	One wire to the pin, the other to GND
L5	ESP32 pins	HX711 DT, SCK, VCC, GND (optional)	26 AWG	Dupont	Load cell 4 wires into HX711 E+, E-, A+, A-
L6	BTS7960 B+	ESP32 battery pin through the 100 k / 22 k divider	26 AWG	solder the 100 k resistor right at the B+ screw (in heat shrink), so the long wire only carries the divided voltage; the 22 k goes from the pin to GND at the ESP32	12.6 V reads 2.27 V at the pin

Gauges assume runs under 0.5 m. 14 AWG carries the 18 A wiper stall; 18 AWG is fine for the Jetson (about 2.5 A at 10 V); 22 AWG is fine for the drum branch (N20 stall about 0.2 A) and the RC832 (about 0.2 A).

4. First power-on sequence (do it in this order, once, after any rewiring)

Before you start: the pull-down and divider resistors are fitted as in firmware/base/WIRING.md (10 k from R_EN to GND, 10 k from L_EN to GND, the IS divider, the battery divider). Both motors are disconnected from their drivers for this first power-on, and the DRV8833 VM wire (P5) is soldered to the DRV8833 but NOT yet to MP1584 #2 OUT (tape its free end). Multimeter ready.

1. Battery unplugged. Jetson barrel plug and ESP32 USB cable unplugged. Pull both fuses (20 A and 10 A) and both small inline fuses.
2. Battery unplugged. Meter on beep, check for shorts: the two pins of the Y harness male XT60; red to black at the BTS7960 B+ / B-; at MP1584 #2 IN and OUT; at DRV8833 VM / GND; at the barrel plug; at the RC832 lead. Nothing may beep steadily. A short chirp that stops is a capacitor charging, not a short. A steady beep is

a short: find it before going on.

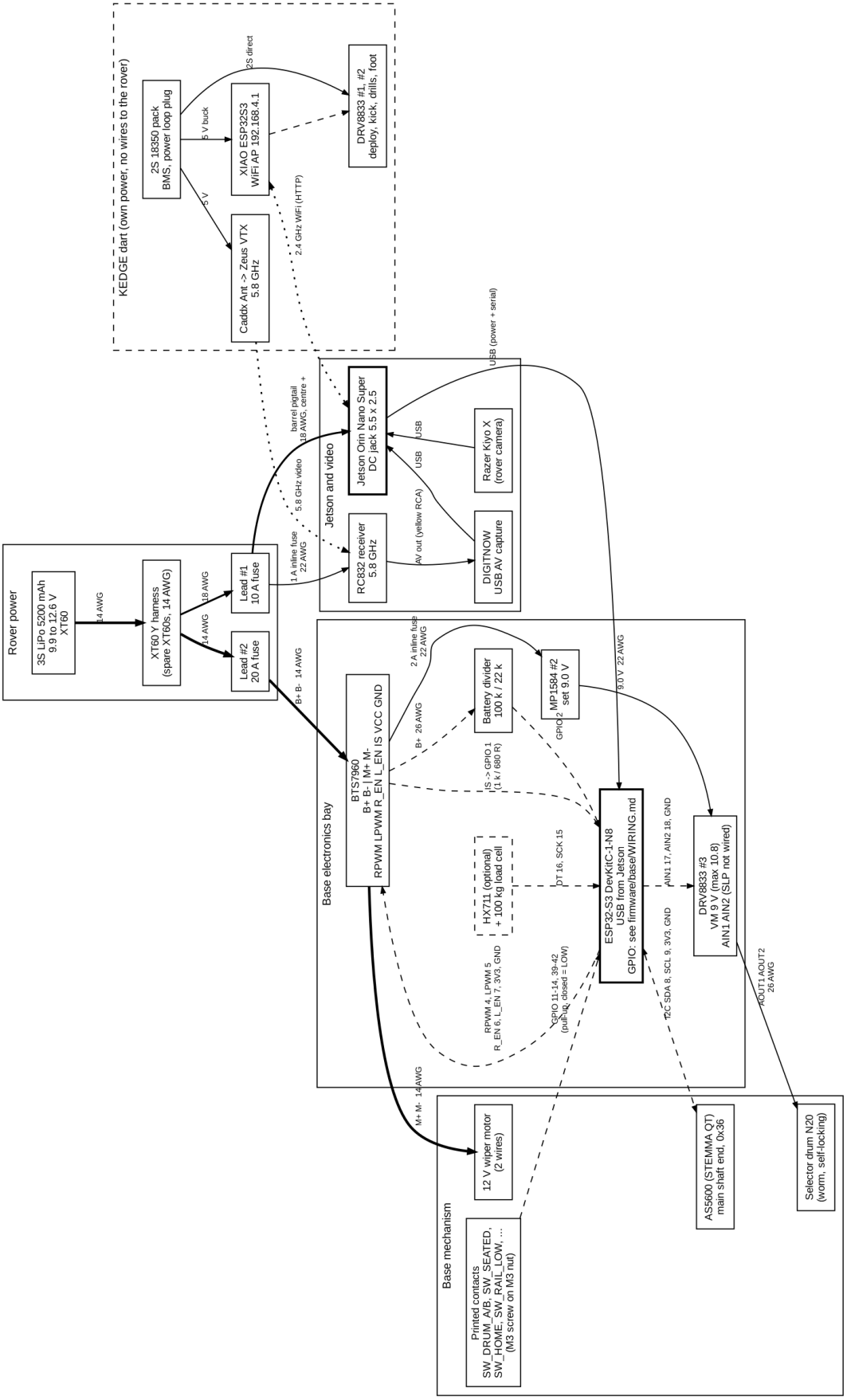
3. Battery unplugged. Look at the wiring: only three DRV8833 #3 pins go to the ESP32 (AIN1, AIN2, GND). Its SLP / nSLEEP / EEP pad is NOT wired: the board holds it high by itself. Check no wire is on SLP, VM or VCC going to the ESP32.
4. Battery unplugged. Meter on DC volts on the LiPo balance lead: each cell 3.7 to 4.2 V.
5. Battery unplugged. Fit the 20 A fuse and the 2 A inline fuse. Plug the battery in. Meter on DC volts:
   - BTS7960 B+ (red probe) to B- (black probe): battery voltage, positive. Negative means reversed: unplug at once.
   - MP1584 #2 IN+ to IN-: battery voltage, positive.
   - MP1584 #2 OUT: turn the brass screw until it reads 9.0 V.
6. Unplug the battery. Connect the DRV8833 VM wire (P5) to MP1584 #2 OUT.
7. Battery unplugged. Fit the 10 A fuse and the 1 A inline fuse. Plug the battery in. Meter on the barrel plug (before it goes into the Jetson): centre pin positive, battery voltage. Then plug the barrel into the Jetson: it boots (green LED). The RC832 lights.
8. Plug the ESP32 USB cable into the Jetson (USB is 5 V and safe to plug live). The ESP32 power LED lights.
9. On the Jetson, in `~/GrappStar/jetson`: `python3 -m grappstar base ping` (expect `"ok": true`), then `python3 -m grappstar base monitor`. Check `vbat` in telemetry matches the meter within 0.3 V.
10. Unplug the battery. Connect the motors (wiper to BTS7960 M+ / M-, drum N20 to AOUT1 / AOUT2). Check the B- screw is tight. Plug the battery in. Nothing moves until a command is sent.

Emergency stop: send `estop`. If that does not stop it, pull lead #2's XT60 (or its 20 A fuse): that kills both motors and leaves the Jetson running. Pull the main battery plug only if that is not enough.

Shut-down order (every time): `estop`, then `sudo shutdown now` on the Jetson and wait for its LED to go off, then unplug the battery.

5. Rules

- Battery unplugged before you touch, add or move any wire, or fit a fuse. This killed the first XIAO.
- Battery unplugged before you plug the ESP32 into the Mac. Flash from the Mac with the ESP32 out of the rover harness, or flash from the Jetson (firmware/base/README.md).
- Charge the LiPo at 3.6 V per cell at rest (10.8 V). Never run it below 3.3 V per cell under load (9.9 V); the firmware reports `vbat`.
- Never connect the DRV8833 VM to the LiPo directly (10.8 V maximum; the LiPo is 12.6 V full).
- Never put 3V3 on a DRV8833 pad called VIN, VCC or VM: on these carriers that is the motor supply. On the base, the DRV8833 SLP / nSLEEP / EEP pad is left unwired.
- Dart: power loop plug out whenever a wire moves or USB is plugged in (the dart run sheet rule).
- The VTX on the dart must have its antenna on before it is powered.



Whole system electrical diagram (bold = battery power, dashed = signal, dotted = radio)

7.2 Base ESP32 wiring (pin by pin)

NOT IN THE REPO YET: `firmware/base/WIRING.md` *(written by fw-base). Re-run* `python3 docs/build/make_guide.py` *after it lands and this section fills in.*

8. Flashing both ESP32s

Rules for both boards: battery unplugged (dart: power loop plug out) before the USB cable goes in. Use a data USB cable. Flash the dart XIAO before it goes into the hull (run sheet section 5).

8.1 Base: ESP32-S3 DevKitC-1-N8

> *NOT IN THE REPO YET:* `firmware/base/README.md` *(written by fw-base). Re-run* `python3 docs/build/make_guide.py` *after it lands and this section fills in.*

8.2 Dart: XIAO ESP32S3

Arduino IDE: board XIAO_ESP32S3, Tools > USB CDC On Boot: Enabled (the sketch refuses to compile otherwise). Library: Adafruit BNO08x (the INA219 library is no longer needed). Open `firmware/dart/kedge_dart/kedge_dart.ino`. After the first USB upload you can upload over WiFi (port "kedge").

> *NOT IN THE REPO YET:* `firmware/dart/README.md` *(written by fw-dart). Re-run* `python3 docs/build/make_guide.py` *after it lands and this section fills in.*

9. Jetson setup

The Jetson already runs JetPack 6.2 (Ubuntu 22.04, Python 3.10) from the SD card with WiFi and SSH. Keep numpy below 2 and do not pip install opencv-python (both broke things in August).

From `jetson/README.md`*:*

GrappStar rover software (Jetson)

This folder is the software that runs on the rover's Jetson Orin Nano. It talks to the base station (USB serial), the KEDGE dart (WiFi), and two cameras, and it serves the operator page in a web browser.

Run everything from this folder (`jetson/`) with `python3 -m grappstar <command>`. Every command has `--help`, for example `python3 -m grappstar serve --help`. If this README and `--help` disagree, `--help` is right (and please fix this README).

Marked **NOT VERIFIED** below: things nobody has checked on the real hardware yet.

Contents

0. What you need
1. Get the code
2. Install
3. Test with no hardware
4. Plug in the base
5. Cameras
6. Dart WiFi (and how the operator connects)
7. Calibrate the rover camera
8. Operator page tour and a full dry mission in sim
9. First real mission checklist
10. Troubleshooting
11. File map

Command summary:

Command	What it does
`python3 -m grappstar serve`	operator page at `http://<jetson>:8080`. Options: `--sim`, `--port 8080`, `--base /dev/grappstar-base`, `--dart 192.168.4.1`, `--rover-cam /dev/grappstar-rovercam`, `--dart-cam /dev/grappstar-dartcam`, `--fourcc YUYV` or `MJPG`
`python3 -m grappstar cameras`	list cameras and the formats they offer
`python3 -m grappstar base status` / `ping` / `monitor`	talk to the base. `monitor` prints telemetry live (Ctrl+C stops)
`python3 -m grappstar base cmd '<json>'`	send one raw command, e.g. `'{"cmd": "status"}'`
`python3 -m grappstar dart status`	one dart status reading
`python3 -m grappstar dart key <c> --hold <s>`	hold a dart motor key for s seconds
`python3 -m grappstar dart auto` / `release` / `flightlog`	auto anchor, release sequence, download the flight log
`python3 -m grappstar mockbase --pty`	fake base for testing; prints a pty path to use as `--base`
`python3 -m grappstar calibrate intrinsics` / `mount`	rover camera calibration (step 7)
`python3 -m grappstar replay <logdir>`	re-run the vision on a recorded mission
`python3 -m grappstar selftest`	run checks that need no hardware

---

0. What you need

- Jetson Orin Nano Super with JetPack 6.2 on its SD card (already done in August: hostname `GrappStar`).
- Internet for the Jetson during install (home WiFi or Ethernet).
- A way to type on it: monitor and keyboard, or SSH from the Mac: `ssh <your user>@GrappStar.local` (or the IP address, which was 192.168.0.249 on home WiFi in August).
- For later steps: the base station (ESP32-S3 DevKitC-1) and a USB cable, the Razer Kiyo X, the RC832 receiver and the DIGITNOW USB capture dongle, the KEDGE dart, a phone or laptop for the operator page.

- A printed checkerboard for step 7.

Two things learned in August that this kit works around:

- snap is broken on JetPack 6.2. Use `apt` for everything, never `snap`.
- numpy must stay below 2. pip once upgraded it to 2.x and broke system packages. The install keeps it at 1.x.

1. Get the code

On the Jetson:

```
cd ~
GIT_LFS_SKIP_SMUDGE=1 git clone https://github.com/HarryHYu/GrappStar.git
cd GrappStar
git checkout main
cd jetson
```

- `GIT_LFS_SKIP_SMUDGE=1` skips the big caliper photo PDF (145 MB). The Jetson does not need it.
- If `main` has no `jetson/` folder yet (before the build day merge), use the integration branch instead: `git checkout base-v1`.
- To update later: `cd ~/GrappStar && git pull`.

2. Install

```
cd ~/GrappStar/jetson
bash setup/install.sh
```

It asks for your password (sudo). What it does, in order:

1. apt installs `python3-numpy python3-serial python3-pytest python3-pip v4l-utils network-manager`.
2. OpenCV: JetPack normally already has OpenCV for Python (NVIDIA's build, with GStreamer). If `import cv2` already works the script leaves it alone. Only if it is missing does it run `apt install python3-opencv`. Why not `pip install opencv-python`? The pip version has no GStreamer, carries its own JPEG and video libraries (August's JPEG mismatch was exactly this kind of clash), and can drag numpy up to 2.x.
3. `pip install --user -r requirements.txt` for anything apt did not provide. The file pins `numpy>=1.21,<2`.
4. Adds you to the groups `dialout` (serial ports) and `video` (cameras).
5. Installs `setup/99-grappstar.rules` so the devices get fixed names: `/dev/grappstar-base`, `/dev/grappstar-base-uart`, `/dev/grappstar-rovercam`, `/dev/grappstar-dartcam`.
6. Runs `setup/check_env.py` and prints a report.

Then **log out and back in** (or `sudo reboot`) so the new groups work.

Check any time with:

```
python3 setup/check_env.py
```

Look for: numpy 1.x, cv2 with `V4L2 YES` (and `GStreamer YES` on the Jetson), pyserial, groups dialout and video yes.

Optional: start the operator page automatically at boot (do this after step 3 works):

```
bash setup/install.sh --service
systemctl status grappstar          # should say "active (running)"
journalctl -u grappstar -f          # live log, Ctrl+C to leave
```

The service runs `python3 -m grappstar serve --port 8080` as your user from this folder, and restarts it if it crashes. While the service runs, do not also start `serve` by hand (both want port 8080). Stop it with `sudo systemctl stop grappstar`, remove it from boot with `sudo systemctl disable --now grappstar`.

Optional: onnxruntime-gpu for a GPU terrain model. Not needed to run anything. See the note at the bottom of `requirements.txt` (Jetson AI Lab wheel, NOT VERIFIED).

Optional: maximum performance mode (already set in August): `sudo nvpmodel -q` shows the mode. `jtop` shows load.

3. Test with no hardware

Nothing needs to be plugged in for this step. It also works on a PC.

```
cd ~/GrappStar/jetson
python3 -m pytest                 # all tests; ends with "N passed"
python3 -m grappstar selftest     # quick checks, prints PASS or FAIL per check
python3 -m grappstar serve --sim  # operator page with a simulated base, dart and cameras
```

With `serve --sim` running, open the page:

- on the Jetson itself: `http://localhost:8080`
- from the Mac or phone on the same network: `http://GrappStar.local:8080` or `http://<jetson IP>:8080` (find the IP with `hostname -I`).

You should see both camera views moving (simulated) and base and dart status updating. Ctrl+C in the terminal stops the server. Step 8 walks through a whole simulated mission.

Test the real base client against a fake base (two terminals):

```
python3 -m grappstar mockbase --pty          # terminal 1: prints a path like /dev/pts/5
python3 -m grappstar base ping --base /dev/pts/5    # terminal 2, using the path it printed
```

(The `--base` option on the `base` subcommand is assumed to match `serve`; check `python3 -m grappstar base --help`.)

4. Plug in the base

Battery unplugged. For this step the base runs from USB only: the Jetson's USB powers the DevKitC. Do not connect the 3S LiPo, the BTS7960 or the motors yet.

1. Battery unplugged. Plug a USB cable from the Jetson into the DevKitC port marked **USB** (the ESP32-S3 native port), not the one marked UART or COM.
2. Check the name appeared:

```
ls -l /dev/grappstar-base
lsusb                              # expect a line with 303a:1001 (Espressif)
```

`/dev/grappstar-base -> ttyACM0` means it worked. If you used the UART port instead you get `/dev/grappstar-base-uart`; move the cable to the USB port.

3. Talk to it:

```
python3 -m grappstar base ping      # expect a reply with "ok": true
python3 -m grappstar base status
python3 -m grappstar base monitor   # telemetry lines about 10 per second, Ctrl+C stops
```

4. Only after this works, and following the base build guide: battery unplugged, wire the power side, then connect the battery last.

The base firmware must use the native USB CDC (Arduino IDE: "USB CDC On Boot: Enabled", "USB Mode: Hardware CDC and JTAG"). The base talks JSON lines (contract C1, `firmware/base/PROTOCOL.md`).

5. Cameras

Battery unplugged before connecting the RC832 receiver's power. Always screw the antennas onto the RC832 and onto the dart's Caddx Ant VTX before powering them: a VTX powered without an antenna can burn out.

1. Plug in the Kiyo X and the DIGITNOW capture dongle (USB). Connect the RC832 video out (yellow) to the dongle's video in. Battery unplugged while you wire the RC832 power; power it as its label says (NOT VERIFIED which supply on the rover).
2. List the cameras:

```
python3 -m grappstar cameras
ls -l /dev/grappstar-rovercam /dev/grappstar-dartcam
```

If a name is missing, run `lsusb`, find the camera's `ID vvvv:pppp`, and fix the line in `setup/99-grappstar.rules` (the Kiyo X ID 1532:0e4f and the dongle ID 534d:2109 are NOT VERIFIED). Then rerun `bash setup/install.sh --no-apt` and replug.

3. See exactly which formats each camera offers:

```
v4l2-ctl -d /dev/grappstar-rovercam --list-formats-ext
v4l2-ctl -d /dev/grappstar-dartcam --list-formats-ext
```

Look for `YUYV` and `MJPG` and the sizes listed under each.

4. Pick the format. The software always asks for a format explicitly, because on JetPack the default path through NVIDIA's JPEG library failed in August (Cheese crashed). Default is YUYV:

```
python3 -m grappstar serve --fourcc YUYV     # safe, works at lower resolutions
python3 -m grappstar serve --fourcc MJPG     # needed for 1920x1080 on the Kiyo X
```

If the picture is black or you see JPEG errors with one, try the other.

5. Dart video: power the dart (its VTX transmits). On the RC832, press the button to step through channels until the picture is clean. The RC832 must be on the same band and channel as the Caddx Ant VTX (the VTX shows its channel with its LED or button; see its manual). Snow means wrong channel, no antenna, or too far.
6. Start `serve` and check both views on the page. The dart view is analog: some snow and dropouts are normal, and the software marks bad frames and skips them for vision.

6. Dart WiFi (and how the operator connects)

The dart is a WiFi access point: name **KEDGE**, password **grappstar**, address **192.168.4.1**. The Jetson must join it to control the dart. At the same time the operator's phone or laptop must reach the Jetson's page.

The plan

- The Jetson's built-in WiFi joins KEDGE. Always.
- The operator connects in one of three ways. **Start with A** (nothing extra to buy). Move to B or C for real missions if A is slow or drops.

Option	Operator connects to	Page address	Good	Bad
A	KEDGE too	`http://<Jetson KEDGE IP>:8080`	nothing extra, simplest	all video goes through the dart's small ESP32 radio, which also carries the dart's own commands. NOT VERIFIED how well it copes
B	Jetson hotspot GRAPPSTAR-ROVER on a USB WiFi adapter	`http://10.42.0.1:8080`	separate radio for video	needs a USB WiFi adapter with Linux AP mode support
C	travel router plugged into the Jetson's Ethernet	Jetson's address on the router	most reliable	one more box to power

The ESP32 access point allows only a few clients at once (4 by default), so with option A keep it to the Jetson and one operator device.

Join KEDGE from the Jetson

Do the install (step 2) first: once the built-in WiFi is on KEDGE, the Jetson has no internet through it.

Power the dart, then on the Jetson:

```
nmcli device                                  # find the WiFi interface, usually wlP1p1s0
nmcli device wifi list                        # KEDGE should be listed
sudo nmcli device wifi connect KEDGE password grappstar ifname wlP1p1s0 name kedge
# KEDGE has no internet: never use it as the default route
# keep retrying when the dart is off, and prefer it over home WiFi
sudo nmcli connection modify kedge ipv4.never-default yes ipv6.method ignore \
    connection.autoconnect yes connection.autoconnect-priority 10 connection.autoconnect-retries 0
ping -c 3 192.168.4.1
python3 -m grappstar dart status
```

Replace `wlP1p1s0` if `nmcli device` shows a different name. Back to home WiFi for updates: `nmcli connection up <home wifi name>` (list names with `nmcli connection show`). Note: with priority 10, the Jetson picks KEDGE whenever the dart is on.

Dart commands:

```
python3 -m grappstar dart status            # phase, battery, motors
python3 -m grappstar dart key 2 --hold 1.0  # hold "kick out" for 1 s
python3 -m grappstar dart auto              # auto anchor sequence
python3 -m grappstar dart release           # release sequence
python3 -m grappstar dart flightlog         # download the last flight log
```

Dart keys: `1` deploy down, `!` deploy up, 2 kick out, @ kick back, 3 drills in, # drills out, 4 foot in, $ foot out. Quote the symbols in the shell, e.g. `dart key '!' --hold 1.0`.

Option A: the operator joins KEDGE too

1. On the Jetson, find its KEDGE address:
   ```
   ip -4 addr show wlP1p1s0         # look for "inet 192.168.4.x"
   ```
2. Phone or laptop: join WiFi KEDGE (password grappstar). It will say "no internet": choose to stay connected (Android asks; turn off "switch to mobile data" if it keeps leaving).
3. Open `http://192.168.4.x:8080` (the x from step 1). `http://GrappStar.local:8080` may also work (mDNS works on iPhone and Mac, not on every Android).

Option B: hotspot on a USB WiFi adapter

1. Plug in the USB WiFi adapter. `bash setup/hotspot.sh list` shows it (a name like `wlx...`).
2. Start the hotspot (it also starts at every boot from now on):
   ```
   bash setup/hotspot.sh up                # picks the free WiFi interface
   bash setup/hotspot.sh up wlx00c0ca123456 # or name it
   ```

This runs `nmcli device wifi hotspot ifname <adapter> con-name grappstar-hotspot ssid GRAPPSTAR-ROVER password grappstar`.

3. Operator: join **GRAPPSTAR-ROVER** (password grappstar), open `http://10.42.0.1:8080`.

4. Stop it: `bash setup/hotspot.sh down`. Status: `bash setup/hotspot.sh status`.

Do not run the hotspot on the built-in WiFi: that radio is on KEDGE.

Option C: travel router on Ethernet

1. Set up the travel router as a normal WiFi router (its own WiFi name and password, DHCP on). Plug a cable from one of its LAN ports to the Jetson's Ethernet port.
2. On the Jetson:

```
nmcli device                                    # find the Ethernet name, usually eth0 or enP8p1s0
sudo nmcli connection add type ethernet ifname eth0 con-name rover-lan ipv4.method auto \
    connection.autoconnect yes
ip -4 addr show eth0                            # the Jetson's address on the router
```

Replace `eth0` with the name `nmcli device` showed.

3. Operator: join the router's WiFi and open `http://<that address>:8080`. In the router's admin page you can reserve that address so it never changes.

7. Calibrate the rover camera

The rover aims the base at things it sees, so it must know how its camera maps pixels to angles (intrinsics) and how the camera sits on the rover (mount: height and tilt).

The exact options of the `calibrate` commands are set by the code: run `python3 -m grappstar calibrate intrinsics --help` and `python3 -m grappstar calibrate mount --help` first. The steps below are the method.

Print the checkerboard

- **9 by 6 inner corners** (that is 10 by 7 squares), **25 mm squares**. A generator such as calib.io ("Checkerboard", 10 columns, 7 rows of squares, 25 mm) or OpenCV's `pattern.png` works.
- Print at 100 percent ("actual size", no "fit to page"). Measure a square with calipers. If it is not 25.0 mm, give the measured size to the command (see `--help`).
- Tape it flat on something stiff (cardboard, a clipboard). A bent board gives bad results.

Intrinsics

1. Stop the operator page if it is running (it holds the camera).
2. Run `python3 -m grappstar calibrate intrinsics` (with `--help` options for the camera and square size).
3. Hold the board in front of the camera and move it: near and far, left, right, top, bottom corners, and tilted. Aim for 20 or more good views. Keep it still for each capture and avoid glare.
4. It prints the result (and an error number: under 0.5 pixels is good, over 1 means redo it) and where it saved the file.

Mount

1. Fix the camera in its final position on the rover. If it moves later, redo this part.
2. Rover on flat ground. Lay the board flat on the ground in front of it, in view.
3. Run `python3 -m grappstar calibrate mount` and follow what it asks (measured distances, see `--help`).
4. It saves camera height and pitch.
5. Check: put an orange object at a distance you measured with a tape (for example 5.0 m), click it on the page, and compare the range the page shows. Within about 5 percent is good.

8. Operator page tour and a full dry mission in sim

Start the simulator and open the page (step 3):

```
python3 -m grappstar serve --sim
```

What is on the page (layout may differ a little from this list):

- **Camera views**: rover camera and dart camera, live. Click on the rover view to mark a target.
- **Base panel**: link, state, mode, drum position, aim angle, current, battery, fault.
- **Dart panel**: link, phase (IDLE, ARMED, FLIGHT, LANDED, ANCHORING, ANCHORED, RELEASING, FAULT), battery.
- **Mission panel**: current step, history, and the **prompt** box. Before anything dangerous (firing, reeling) the mission stops and asks you to confirm, e.g. "Fire" or "Cancel". Nothing dangerous happens without your click.
- **Target**: range and bearing of the clicked or auto found target, and the ballistic solution (pitch, flight time).

- **Results**: after a flight, the landing zone verdict (GO, NO_GO or UNSURE with reasons), the landing image, the aerial map built from the dart video, and the flight analysis.
- **Manual**: jog the wiper and drum motors, dart keys, record on or off.
- **STOP** (wiper off now) and **E-STOP** (wiper off and drum to neutral). Always visible. Know where they are before you press anything else.

Dry mission in sim, step by step:

1. Press **Preflight**. Everything should go green (base link, dart link, cameras).
2. Click a target on the rover view (or press auto target). Check the range and solution.
3. Press **Start**. Answer each prompt. At the fire prompt press **Fire**.
4. Watch the dart view: the simulated flight, impact and landing.
5. Read the landing verdict and look at the map.
6. Follow the prompts through anchoring, tensioning, release and reeling in.
7. Try **STOP** and **E-STOP** in the middle of a second run and check the mission stops and says why.
8. Replay the recorded mission's vision:

```
python3 -m grappstar replay <logdir>
```

where `<logdir>` is the mission log folder (its path is printed in the terminal and shown on the page when the mission ends).

9. First real mission checklist

Before the first real launch:

- [ ] Steps 3 to 8 all worked. `python3 -m pytest` passes on the Jetson.
- [ ] Clear area: nobody in front of the launcher or anywhere along the flight path or the line. Check behind the target too. Everyone knows the shout for stop.
- [ ] Line attached at both ends (dart and reel) and laid so it cannot snag on the rover, feet or wires.
- [ ] Tension limit set to a low value for the first runs (the base `set` key, or the page) and you know what it is.
- [ ] STOP button: you can see it on the page, and your hand is on it during every motion. Know that E-STOP sends the drum to neutral. Know where the battery connector is so you can unplug it fast.
- [ ] Battery charged (3S full is 12.6 V), 20 A fuse fitted. Battery unplugged while you check any wiring.
- [ ] Antennas on the VTX and RC832 before power. Dart view clean on the page.
- [ ] Dart status OK (`python3 -m grappstar dart status`), base ping OK, preflight all green.
- [ ] Recording on, so you can replay it.
- [ ] Start with a short shot at low pitch.

10. Troubleshooting

Problem	Likely cause	Fix
No `/dev/grappstar-base`	rules not installed, cable in the UART port, charge-only cable, or a different USB ID	`ls /dev/ttyACM* /dev/ttyUSB*`, `lsusb` (expect 303a:1001). Use the DevKitC port marked USB and a data cable. Rerun `bash setup/install.sh --no-apt` and replug. If lsusb shows another ID, edit `setup/99-grappstar.rules`
Only `/dev/grappstar-base-uart`	cable in the UART/COM port	move it to the port marked USB
`/dev/ttyUSB0` appears then vanishes (CH340)	brltty grabs it	`bash setup/install.sh --remove-brltty`
`Permission denied` on the serial port or camera	not in groups dialout or video yet	`groups` should list both. If not, `bash setup/install.sh --no-apt`, then log out and in (or reboot)
Base ping times out	wrong firmware USB mode, or another program has the port	check the firmware is built with "USB CDC On Boot: Enabled". Stop the service (`sudo systemctl stop grappstar`) or other `serve` before using `base` commands
Camera black, or JPEG errors like "Corrupt JPEG data"	format mismatch with NVIDIA's JPEG library (seen in August)	set the format explicitly: `--fourcc YUYV`, or `--fourcc MJPG` if YUYV does not offer the size. Check sizes with `v4l2-ctl --list-formats-ext`
Camera "busy" or will not open	another program has it (Cheese, a second `serve`, the service, calibrate)	close it. `sudo fuser /dev/video*` shows who holds it

Problem	Likely cause	Fix
Dart view is snow	RC832 channel does not match the VTX, antenna missing, dart off or too far	step the RC832 channel, check both antennas, move closer
Dart view stuck or blue screen	RC832 has no signal, dongle not getting video	check the yellow video cable and RC832 power
Dart unreachable (`dart status` fails, no ping to 192.168.4.1)	Jetson not on KEDGE, dart off, or out of range	`nmcli device` (is the WiFi on `kedge`?), `nmcli connection up kedge`, power the dart, move closer
Page unreachable from the phone	phone on a different network, wrong address, or serve not running	phone must be on KEDGE (A), GRAPPSTAR-ROVER (B) or the router (C). Find the address with `hostname -I`. On the Jetson, `curl -I http://localhost:8080` checks serve is up. `systemctl status grappstar` if using the service
Phone keeps leaving KEDGE	it wants internet	tell it to stay connected; turn off mobile data switching; or use option B
`numpy.core.multiarray failed to import`, "compiled using NumPy 1.x cannot be run in NumPy 2"	numpy 2 got installed by pip	`python3 -m pip install --user 'numpy>=1.21,<2'`, then `python3 setup/check_env.py`
Two OpenCVs (cv2 version or GStreamer NO in check_env on the Jetson)	`pip install opencv-python` was run	`python3 -m pip uninstall opencv-python opencv-python-headless`, keep the apt one
A program wants `snap install`	snap is broken on JetPack 6.2	use apt (or the project's apt or pip instructions) instead
Port 8080 already in use	the service is running, or an old `serve`	`sudo systemctl stop grappstar`, or close the other terminal. Or use `--port 8081`

11. File map

```
jetson/
  README.md              this guide
  INTERFACES.md          how the modules fit together (for programmers)
  requirements.txt       Python packages (numpy below 2, pyserial, pytest; OpenCV from apt)
  pytest.ini             test settings (tests/ folder)
  setup/
    install.sh           installs everything (step 2)
    check_env.py         prints what is installed and plugged in
    99-grappstar.rules   udev rules: /dev/grappstar-base, -base-uart, -rovercam, -dartcam
    grappstar.service    systemd unit to start the page at boot
    hotspot.sh           optional operator hotspot on a USB WiFi adapter (option B)
  grappstar/
    __main__.py, cli.py  the command line (python3 -m grappstar ...)
    app.py               owns the base, dart, cameras and mission; what the page talks to
    mission.py           the mission steps and operator prompts
    rover.py             rover side helpers
    config.py            settings and defaults
    jlog.py              mission logs
    ballistics.py        aim solution from the ballistics table
    data/                placeholder ballistics table
    comms/               base.py (serial), dart.py (HTTP), mock_base.py, mock_dart.py
    video/               camera capture, discovery, frame quality, recording, replay
    vision/              flight.py, mosaic.py (aerial map), terrain.py (landing verdict), target.py (
  aiming)
    sim/                 simulated dart flight video and terrain textures
    ui/                  operator web page
  tests/                 one test file per module, run with python3 -m pytest
```

10. Calibration (base)

Do it after the base is assembled and wired, springs off, before test stage 3. The values are stored on the ESP32.

> *NOT IN THE REPO YET:* `firmware/base/CALIBRATION.md` *(written by fw-base). Re-run* `python3 docs/build/make_guide.py` *after it lands and this section fills in.*

11. Bench tests

11.1 Base

NOT IN THE REPO YET: `firmware/base/BENCH_TEST.md` *(written by fw-base). Re-run* `python3 docs/build/make_guide.py` *after it lands and this section fills in.*

11.2 Dart

The dart bench tests are in section 6 (`firmware/dart/BENCH_TEST.md` and the dart build order, steps 6, 7 and 11). Foot check: key 4 turns the foot auger clockwise seen from the motor end, looking toward the tip.

12. First tests, bench to field

From `docs/build/TEST_PLAN.md`*:*

First tests: bench to field

Do the stages in order. Do not skip ahead when a stage fails: fix it, then repeat the stage. Write the date and the result of each stage in a notebook (or `docs/build/test_log.md`).

How to send base commands: the operator page on the Jetson (jetson/README.md) has buttons for most of these. From a terminal on the Jetson, in `~/GrappStar/jetson`, send any command (firmware/base/PROTOCOL.md) with `python3 -m grappstar base cmd '{"cmd": "jog", "motor": "wiper", "dir": 1, "ms": 300}'` and watch telemetry with `python3 -m grappstar base monitor` in a second terminal. `'{"cmd": "estop"}'` stops everything and is always safe to send.

Safety for every stage:

- Battery unplugged before you touch any wire.
- Hands out of the gears, the drum and the rail whenever the battery is plugged in.
- From stage 5 on: nobody in front of the rail, safety glasses on, the launch area clear for 30 m.
- Emergency stop: `estop`. If that does not stop it, pull lead #2's XT60 (the wiper branch): both motors die and the Jetson stays up. Keep a hand near it.

Stage 0. Power-on (no motors attached)

Follow SYSTEM_ELECTRICAL.md section 4, the first power-on sequence, steps 1 to 9. Both motors stay disconnected from their drivers until step 10.

Pass: no steady beeps on the short check, MP1584 #2 reads 9.0 V, the Jetson boots, `ping` replies, `vbat` within 0.3 V of the meter, `fault` is null. If it fails: a beep means a short (check the Y harness and the BTS7960 screws first). No reply: check the USB cable is a data cable and in the ESP32 port marked USB; see troubleshooting in the build guide.

Stage 1. Each motor alone, out of the base

Battery unplugged. Connect the wiper motor to BTS7960 M+ / M- and clamp the motor to the bench. Connect the drum N20 to DRV8833 #3 AOUT1 / AOUT2, loose on the bench. Battery in.

1. `jog` wiper dir 1 for 300 ms, then dir -1. Mark with a pen arrow on the motor which way the shaft turns for dir 1. That is FORWARD (the cock direction); CALIBRATION.md says how to make it match the base.
2. `jog` drum dir 1 for 500 ms, then dir -1. Mark the shaft direction.
3. Do the current-sense calibration now (MEASURE_LIST G1, CALIBRATION.md), then watch `amps` during a 2 s wiper jog.

Pass: each motor turns both ways, stops when the jog ends, wiper free current about 1 to 3 A (A6 in MEASURE_LIST.md), no fault. If it fails: no movement on the wiper: check R_EN / L_EN wiring and the 20 A fuse. Wiper turns only one way: one of RPWM / LPWM is not connected. Drum does nothing: check 9.0 V on VM and the AIN1 / AIN2 / GND jumpers. If it still does nothing, the board's SLP may not be pulled high on its own (some non-Pololu boards): battery unplugged, add a wire from DRV8833 SLP to the ESP32 3V3 pin (never to VM). Wrong direction against CALIBRATION.md: swap that motor's two leads (battery unplugged).

Stage 2. Sensors alone

1. Turn the main shaft (or the magnet) by hand: `wiper_deg` changes smoothly, no jumps, and goes up when the shaft turns the FORWARD way (the arrow you drew in stage 1).
2. Touch each printed contact closed with a finger: its entry in `sw` changes and changes back when released.
3. Load cell (if fitted): press it, `tension_n` rises.

Pass: every sensor responds and returns. If it fails: AS5600 not found: check SDA / SCL are not swapped and the STEMMA cable is pushed in; angle jumps: magnet too far (keep 1 to 3 mm) or off centre. Contact never changes: check the GND wire and that the screw head really touches the nut.

Stage 3. Base assembled, dry cycles, NO springs, no line

Springs off, no tether, no dart. Run CALIBRATION.md first (zeros and drum positions).

1. `mode` LAUNCH, NEUTRAL, REEL, NEUTRAL, LAUNCH: 20 full rounds. Each change ends with the right `drum_pos`.

2. `aim` 15, 30, 45, then the minimum and maximum pitch: 5 rounds. Rail reaches each angle within 1 degree (`pitch_deg`) and holds with the motor off.
3. Cock and fire with no springs: `arm`, then `fire` with the token. The carriage travels back, releases, and the big gear re-engages: 10 cycles.
4. REEL mode: `wrap`, then `unwrap`: 10 cycles. Then `reel` at duty 0.3 for 3 s with no line.
5. Send `estop` in the middle of each kind of move once. The wiper stops at once and the drum goes to NEUTRAL.
6. Fault test: send `estop`, then unplug the AS5600 STEMMA cable (it is 3.3 V from USB; the motor branch is not touched). The base reports `ANGLE_SENSOR` and refuses to move. Plug it back in, `clear`.

Pass: all cycles complete with no fault, no clicking or skipping teeth, no part warm to the touch, the drum never stops BETWEEN positions. If it fails: a STALL fault means something binds: find it by turning by hand with the battery unplugged. Skipped teeth: check the mesh spacing and the frame screws. Drum stops BETWEEN: re-run the drum calibration; check the drum contacts close cleanly.

Stage 4. Springs on, no dart

Fit both springs. Start at the lowest pitch, rail pointing at a clear area.

1. `aim` to the lowest pitch. `arm` and `fire` once. Look at every part of the launch train after the shot.
2. Five more shots. Check the stop bumpers, the carriage, the rack teeth and the big gear's last tooth after each.
3. Watch `amps` while cocking: note the peak.

Pass: the carriage stops on the bumpers, nothing cracks or loosens over 6 shots, cocking current stays under the stall value, the rail holds its pitch while cocking. If it fails: cracks at the bumpers: stop and report (no more dry shots). Rail drops while cocking: the aim worm is slipping, check its mesh. STALL while cocking: the springs are stronger than planned (redo B2) or something binds.

Stage 5. Dummy dart, no tether

Make the dummy: the plate A hull top and bottom, nose and tail cap (spare prints or the dart shells before the electronics go in), taped shut with M8 nuts or sand inside to 280 g. Paint the nose bright.

1. `aim` 20 degrees. Load the dummy, check `SW_SEATED` is closed. Fire.
2. Film it side-on with the phone in slow motion, tape measure on the ground in view (muzzle speed, G5).
3. Measure the range. Repeat at 30 and 45 degrees (G6).

Pass: clean exit (no rubbing marks on the dummy), it flies nose first, range within 30 percent of the physics model's no-tether range (`base/physics/SUMMARY.md`, "Same shot, no tether": 5.2 m at 35 degrees before the springs were measured). If it fails: tumbles: check the nose ballast is in the dummy too (it must match the real dart). Short range: measure the spring (B2) and the muzzle speed, then re-run the physics. Rubbing marks: check the cradle and the barrel against the dart.

Stage 6. Tether on

Tie the line through the dummy's tail cap holes. Line routed as in the base assembly steps (it bypasses the capstan for launch). Storage reel brake set with hanging weights (G4): 50 g does not pull line off, 75 g does.

1. Fire at 30 degrees.
2. Check the line paid out with no loops or overrun on the reel.
3. `mode` REEL, `wrap`, then `reel` until `SW_SEATED` or the dummy is back near the base.
4. `unwrap`, `mode` LAUNCH. Check the line runs on the launch path again.

Pass: payout is clean, the range is at least 65 percent of stage 5 at the same pitch (the physics model expects about 72 percent: 3.8 m against 5.2 m at 35 degrees), the wrap lays on the capstan, reel-in works, unwrap returns the line to the launch path. Repeat 5 times. If it fails: backlash loops on the reel: raise the payout drag a little (do not exceed 1 N). Big range loss: the drag is too high or the line catches; watch the slow-motion video. Wrap misses the capstan: check the wrap arm position at the end of `wrap`.

Stage 7. The anchor in soil

The real dart, built and tested to the dart run sheet section 10. Use real soil (moist loam is the target).

1. Place the dart on the soil by hand, tether tied, line slack. Send `/auto` from the operator page (or key a).
2. When the dart reports ANCHORED: pull the tether by hand with the luggage scale, slowly, up to 50 N or until it moves (G7).
3. Re-anchor. Base in REEL with wraps on: `reel` with `until` tension at a limit 10 N under the hand result.

4. Send release (operator page, or `python3 -m grappstar dart release`): drills out, pods up, foot out, kick back. Then reel the dart home.

Pass: the dart anchors on its own, holds at least 20 N in loam, the base stops at its tension limit, the release sequence frees the dart, it reels home. If it fails: augers back out: swap that motor's leads (dart run sheet section 10). Pulls out at low force: try firmer soil, and report the force (it sets the reel limit). Base overshoots the limit: redo the current calibration (G1).

Stage 8. Full mission

From the operator page on the Jetson (jetson/README.md): preflight, pick a target on the Kiyo view, aim, confirm the fire prompt, flight video from the dart, landing verdict, anchor, tow, release, reel home.

1. First run: a short target well inside the range the operator page offers (the physics model gives about 4 m at 35 degrees with the current springs), soft soil, open ground.
2. Watch every prompt: the software stops and asks before each dangerous step.

Pass: the whole sequence runs with only the expected operator confirmations, the dart lands within 2 m of the target, anchors, the rover is pulled toward the dart, the dart releases and is reeled home. If it fails: note the mission step and the fault code in the log (`mission.jsonl` on the operator page), then repeat the stage where that step was first tested.

Expected numbers (physics model)

Compare your test results with these. They are recomputed from params.py, so after you enter the measured spring and wiper numbers and re-run the physics, they update.

From `base/physics/SUMMARY.md`*:*

Base station physics: summary

Params: base/spec/params.py v0.1-stub (still defaults for LAUNCH, AIM, REEL, LAYOUT). Generated by base/physics/summary.py.

Spring (each, A228, SF 1.2 on Shigley static limits)

Item	Value
Rate used / by formula	0.125 / 0.128 N/mm
Initial tension used	7.0 N
Body coils, mass	98, 55 g
Safe force (governed by)	43.6 N (hook torsion)
Safe extension (wire) / spec cap	293 / 250 mm
Surge (fixed-fixed)	24 Hz

Launch (chain spring model)

Item	Value
Stroke, ratio, pitch	240 mm, 1.0:1, 35 deg
Spring extension full / rest	250 / 10 mm
Stored energy (2 springs)	11.2 J
Carriage force at full draw	76 N
Muzzle speed	7.13 m/s
Peak acceleration, stroke time	20 g, 56 ms
Efficiency (dart KE / stored)	63 %
Carriage KE at stop, pad peak	1.52 J, 762 N (4 mm pad)
Spring KE at stop, coil clash	0.99 J, 10.5 m/s

Flight (bead-chain tether, level ground)

Item	Value
Range at 0.5 / 0.75 / 1.0 N brake	4.20 / 3.77 / 3.40 m (pitch 35)
Same shot, no tether	5.22 m
Flight time, apex (0.75 N)	0.81 s, 1.07 m
Impact speed, angle (0.75 N)	5.8 m/s, 54 deg down
Ballistics table	15 to 55 deg, best 35.0 deg: 3.96 m

Storage reel payout (0.75 N at full reel)

Item	Value
Reel empty mass, inertia full	35 g, 19.4 kg mm2
Line radius full / core	27.4 / 20.0 mm (brake +37 % at core)
Line pull in the stroke (spin-up)	4.00 N
Reel rpm max	2483 rpm
Overrun after landing	0.23 m in 0.12 s

Capstan and tow

Item	Value
Wraps for tow limit (mu 0.15)	3.4 raw, 6 design
Take-up needed (design wraps)	0.45 N
Tow at limit: torque, speed, current	1.19 N m, 0.077 m/s, 2.80 A
Line pull at wiper stall (12.6 V)	597 N (break 667 N)

Drivetrain (wiper at 11.1 V with sag)

Item	Value
aim: peak torque, current, time	0.19 N m, 1.7 A, 5 s
cock: peak torque, current, time	1.29 N m, 2.9 A, 4 s
reel: peak torque, current, time	0.08 N m, 1.6 A, 235 s
tow: peak torque, current, time	1.19 N m, 2.8 A, 65 s
Wiper stall 12.6 V (with sag)	14.1 N m, 17.0 A

Harry must measure

- Spring hang test: hang 1, 2, 3 kg from one spring, measure the length each time. Slope = rate, intercept = initial tension. Every range number scales with the stored energy.
- Wiper: free speed (turns in 30 s at 12.0 V), free current, stall torque (arm + luggage scale, 1 s) and stall current. Edit common.Wiper or spec WIPER.
- Dart mass on a kitchen scale with ballast and tether knot (spec 0.279 kg).
- Slip clutch: pull the line off the reel with a luggage scale; set 0.5 to 0.75 N.
- Rail friction: tilt the rail with the dart on it until it slides (mu = tan angle).

13. Troubleshooting

First move for any problem: send `estop` (or pull the battery plug), then read the `fault` field in telemetry.

Base

Symptom	Likely cause	Fix
No reply from the base on the Jetson	Charge-only USB cable; cable in the UART port; wrong serial device	Data cable in the port marked USB. On the Jetson, `ls -l /dev/grappstar-base` must show a link to ttyACM0 (jetson/README.md section 4)
ESP32 resets when the wiper starts	Ground or USB power dip	Check the ESP32 GND wire to the BTS7960 GND; short USB cable; report BROWNOUT faults
Wiper does nothing, no fault	R_EN / L_EN not wired, 20 A fuse blown	Battery unplugged, check the fuse and the two enable wires
Wiper turns one way only	RPWM or LPWM loose	Re-seat the Dupont jumper
20 A fuse blows	Stall into a hard stop, short on the wiper leads	Find the bind by turning by hand; never fit a bigger fuse
Fault STALL or OVERCURRENT	Something binds, springs stronger than planned	Battery unplugged, turn the train by hand, find the tight spot; redo the spring measurement
Fault DRUM_TIMEOUT	Drum N20 not turning: no 9 V, board asleep, gear slipping	Meter on DRV8833 VM (9.0 V); check AIN1 / AIN2 / GND; if the board does not pull SLP high by itself, wire SLP to ESP32 3V3 (battery unplugged); check the N20 gear set screw
Fault DRUM_SENSOR or SWITCH_DISAGREE	A drum contact does not close or stays closed	Watch `sw` while turning the drum by the hand lever (battery unplugged for the lever, then plug in to read); bend or re-screw the contact
Fault ANGLE_SENSOR	AS5600 cable out, magnet too far	Re-seat the STEMMA cable; magnet 1 to 3 mm from the chip, centred
`pitch_deg` wrong after a restart	Pitch reference lost	Run the pitch part of the calibration again (section 10)
Drum stuck between positions with the power off	Expected: the worm holds it	Use the hand lever on the drum shaft end, battery unplugged
Base cocked and you need to unload without firing		Clear the area and fire it at the lowest pitch with no dart. If the base cannot fire, battery unplugged, and follow the unload step in the base assembly steps; never hand-release the carriage

Rover and video

Symptom	Likely cause	Fix
Jetson turns off under load	Battery low, or voltage dip	Charge the LiPo; check the barrel plug is tight; check `vbat`
No dart video	VTX and RC832 on different channels; RC832 not powered	Set the RC832 to the VTX channel (scan button); check its LED and the RCA lead
Dart video is snow	Range, antenna off, bad channel	Antenna on both ends; try another channel
Kiyo X shows nothing or errors	JPEG library mismatch on JetPack	The software forces YUYV or MJPEG (jetson/README.md); unplug and replug

Dart

Symptom	Likely cause	Fix
Serial Monitor blank	USB CDC On Boot disabled	Tools > USB CDC On Boot: Enabled, re-upload
No KEDGE WiFi	XIAO not powered or crashed	Loop plug in? 5 V at the XIAO? Press reset
A motor turns the wrong way	Leads reversed	Loop plug out, swap that motor's pair (run sheet section 10)
Current trip during anchoring	Stall on a stone, or the trip set too low	Re-measure stall currents (section 14 F3) and update the tuning
INA219 or BNO085 not found	I2C wiring	`z` key scans; SDA and SCL must not be swapped

14. Measure list

From `docs/build/MEASURE_LIST.md`*:*

What to measure or confirm

Everything nobody could know from here. Do group A and B first: the base parts that depend on them should not be printed until those numbers are in `base/spec/params.py` and the pipeline has been re-run (see "After you measure" at the end). Tools: caliper, multimeter, kitchen scale, luggage scale, phone camera, tape measure.

Photo rule: for every caliper reading, take a photo with the caliper on the part and the reading visible. Put the photos in `originals/measure_2026-11/` so the numbers can be checked later.

A. Wiper motor (params.py `WIPER`, set `measured=True` when done)

#	What	How	Changes	Reprint after
A1	Output shaft diameter	Caliper across the round part of the shaft, 3 places, take the smallest	`WIPER.shaft_d`	Wiper shaft coupler (drum parts)
A2	Shaft flats or spline	Flats: caliper across the flats (`shaft_flat`). Spline: count the teeth, caliper the outside and root diameter, photo end-on	`WIPER.shaft_flat` (note a spline in REPORT)	Wiper shaft coupler
A3	Shaft end thread and length	Thread a nut you know (M6, M8) onto it. Caliper the shaft length from the housing face to the tip	`WIPER.shaft_thread`, `WIPER.shaft_len`	Wiper shaft coupler
A4	Mount hole pattern	Lay the motor face down on paper, press so the holes mark it (or trace with a pencil). Then caliper each hole centre from the shaft centre along two square directions and each hole diameter. Photo of the paper with a ruler	`WIPER.mount_holes` as (x, y, dia) from the shaft axis	Motor adapter plate (frame parts)
A5	Housing size	Caliper the gearbox box (length, width, height) and the motor can (diameter, length), and how far the shaft sits from each edge	`WIPER.body_box`	Frame cheeks and guard if they clash (the audit says)
A6	Free speed and free current	After the first power-on and the current-sense calibration (G1, CALIBRATION.md). Motor clamped, nothing on the shaft. Send a 35 s `jog`. Tape flag on the shaft: count turns for 30 s, rpm = turns x 2. Read `amps` and `vbat` from telemetry. (Not with the meter in series: the start current can pass 10 A and blow the meter fuse)	`WIPER.n0_rpm`, `WIPER.free_a`	Nothing; re-run physics
A7	Stall current and stall torque	Do NOT use the meter: stall is about 18 A, over the meter's 10 A range. After base calibration (firmware/base/CALIBRATION.md), use the `amps` telemetry. Torque: clamp a 300 mm bar on the shaft, hook the luggage scale at the end, scale pulling at 90 degrees to the bar, scale anchored to the bench. Jog for at most 2 s at full duty and read the peak (the firmware stall trip may cut it early: note if it does). Torque N m = kg x 9.81 x 0.30. `python3 base/physics/calib.py wiper --v <vbat> --rpm <A6> --i-free <A6> --stall-kg <kg> --arm-mm 300 --i-stall <amps>` prints the whole WIPER block	`WIPER.stall_a`, `WIPER.stall_nm`	Nothing; re-run physics and the ballistics table

B. Launch springs (params.py `SPRING`, set `measured=True` when done)

The two springs are extension springs with initial tension: they need a small force before they stretch at all, so one weight is not enough. Use three loads. Every range number scales with the energy the springs store, so this is the most important measurement.

#	What	How	Changes	Reprint after
B1	Free length L0	Caliper the spring hook to hook (inside of the hooks), no load	`SPRING.free_len`	Nothing
B2	Rate k and initial tension F0	Use water bottles in a shopping bag, weighed on the kitchen scale: about 1, 2 and 3 kg (W1, W2, W3 in kg). Hang each from ONE spring and measure hook to hook (L1, L2, L3). Plot or compute: k (N/mm) = (W3 - W1) x 9.81 / (L3 - L1). F0 (N) = W1 x 9.81 - k x (L1 - L0). The three points should lie on a line; if L1 equals L0 (the spring did not move), drop W1 and add a 4 kg load, but only if L3 - L0 is under 160 mm. Do both springs. Never stretch a spring longer than L0 + 250 mm. Or let the computer fit it: `python3 base/physics/calib.py spring L0 W1:L1 W2:L2 W3:L3` (kg:mm pairs) prints the rate and initial tension	`SPRING.k_n_per_mm`, `SPRING.F0_n`	Nothing unless the stroke changes (physics says); re-run physics and the ballistics table
B3	Wire and outside diameter	Caliper	`SPRING.wire`, `SPRING.od`	Spring anchors only if OD differs by more than 0.5 mm

C. Parts box: confirm you have these (no new purchases)

#	Item	Needed for	If you do not have it
C1	608 bearings (8 to 10)	storage reel, capstan or wrap arm, cam roller	The only purchase: search "608ZZ bearing 8x22x7"
C2	M8 bolts or 8 mm steel rod, and their lengths	axles through the 608s	Printed PPA-CF axles (tell the coordinator, it is a params switch)
C3	Worm N20 count: you need 4 (dart kick, deploy, foot, and the base selector drum)	dart and base	The kedge notes said 2 in hand and one to buy; the August order listed 8 worm N20s. Count them
C4	MP1584 #2 (the spare from the dart build)	drum N20 supply at 9.0 V	No other way to run the DRV8833 from the 3S: tell the coordinator
C5	DRV8833 #3 (freed from the dart by the new pin map)	drum N20	
C6	100 kg load cell + HX711 (August order)	optional line tension reading	Leave it out: the firmware uses wiper current instead
C7	Resistors: 2 x 10 k, 1 x 1 k, 1 x 680 R, 1 x 100 k, 1 x 22 k	BTS7960 enable pull-downs, IS divider, battery divider (firmware/base/WIRING.md)	Any value within 20 percent works for the pull-downs; tell fw-base if the dividers differ
C8	Spare XT60 plugs, 14 AWG wire, two small inline fuse holders with a 2 A and a 1 A fuse	XT60 Y harness; fuses for the MP1584 and RC832 taps	Without the small fuses: use 18 AWG for those taps and keep them under 10 cm
C9	Barrel plug 5.5 x 2.5 mm with lead	Jetson from the LiPo	Run the Jetson from its own supply on the bench; the rover needs one
C10	Yellow RCA lead	RC832 AV out to the DIGITNOW	
C11	USB data cable for the DevKitC, STEMMA QT to Dupont cable	base ESP32, AS5600	
C12	M3 screws (lengths from the base print list), M3 nuts, M3 heat-set inserts	printed contacts, frame joints	

D. Electrical labels to read

#	What	How	Changes
D1	RC832 input voltage	Read the label or the listing. Most take 7 to 24 V. If it says 12 V only, it still runs from the 3S (12.6 V full) but report it	SYSTEM_ELECTRICAL.md run P8
D2	Jetson DC input range	Label on the Jetson's own supply and the NVIDIA dev kit page. The 3S gives 9.9 to 12.6 V in use and sags during a wiper stall. If the minimum is above 10 V, charge earlier (3.7 V per cell at rest)	SYSTEM_ELECTRICAL.md run P7
D3	DRV8833 carrier make	Is DRV8833 #3 a Pololu board (marked pololu, pads VIN, GND, BIN, AIN, nSLEEP) or a generic red board (pads VCC, EEP, ULT)? The base leaves SLP unwired, which relies on the board pulling it high (Pololu does)	firmware/base/WIRING.md, TEST_PLAN stage 1

#	What	How	Changes
D4	DevKitC board version	Printed on the back (v1.0 or v1.1). The RGB LED pin differs; the firmware avoids both	Nothing
D5	Video standard	When the dart video shows on the Jetson, note the size (720 x 480 = NTSC, 720 x 576 = PAL)	jetson config

E. Rover

#	What	How	Changes	Reprint after
E1	Deck size and mounting holes	Tape measure the free deck area; caliper the hole positions and diameters from one corner	params.py `FRAMEWORK` (rover mount)	Base deck / rover mount
E2	Drive train and whether the Jetson drives it	Tell the coordinator what drives the wheels and what controls it	jetson rover module	Nothing
E3	Kiyo X height and tilt on the rover	Tape measure lens to ground; phone level app on the camera for tilt	jetson camera model (target calibration)	Nothing
E4	Base launch height	Tape measure from the ground to the main shaft axis when the base is mounted	physics (ballistics table)	Nothing

F. Dart (from the KEDGE notes, still open)

#	What	How	Changes
F1	Dart v30 is the one being built	Confirm	print pack choice
F2	Bought part weights and the finished dart weight	Kitchen scale	`DART.mass_kg`, dart physics
F3	Stall current of each dart motor at 2S	The dart `i` key while holding the shaft with pliers for 1 s	firmware/dart tuning values (see firmware/dart/README.md)
F4	OTOTEC BMS size and F693ZZ flange	Caliper (pocket is 40.8 x 8 x 2.1 plus 0.5 each way)	dart geometry if it does not fit
F5	Nose ballast	Weigh it: 15 g	flight stability

G. Bench and field numbers (measured during the TEST_PLAN stages)

#	What	How	Changes
G1	BTS7960 current sense scale	firmware/base/CALIBRATION.md (meter in series at low duty, compare with `amps`)	firmware `set` key (stored on the ESP32)
G2	Battery voltage reading	Meter on the LiPo vs `vbat` telemetry	firmware `set` key
G3	AS5600 zero and drum positions	firmware/base/CALIBRATION.md	stored on the ESP32
G4	Storage reel payout drag	A luggage scale cannot read this low: use hanging weights in LAUNCH mode. Tie 50 g (coins in a bag, weighed) to the line end over the edge of the bench: it must NOT pull line off. 75 g must pull line off slowly. That is 0.5 to 0.75 N. Above 1 N costs range, 3 N ruins the flight	slip clutch preload (reel parts)
G5	Muzzle speed	Phone slow-motion video side-on, tape measure laid in the flight line (same distance from the phone as the dart). Count the frames the nose takes to cross 0.5 m; speed m/s = 0.5 x slow-motion fps (240 or 120) / frames	physics model, ballistics table
G6	Range at three pitches	Tape measure from the base to the dart at 3 pitches, no tether brake changes	ballistics table check (jetson uses it)
G7	Soil holding force	Dart anchored, luggage scale on the tether, pull slowly up to 50 N, note where it moves	reel tension limit
G8	Rail friction	Before the rail is mounted: lay the rail part on a flat board, dart in the cradle, lift one end of the board slowly until the dart slides; phone level app on the board. Friction = tan(angle)	physics (launch efficiency)

After you measure

1. Write the numbers into `base/spec/params.py` (set `measured=True` in `WIPER` and `SPRING`) or send them to the coordinator. Put photos in `originals/measure_2026-11/`.
2. Re-run the base pipeline (assembly, audit must say CLEAN, physics, ballistics table, print pack) and `firmware/base/tools/gen_params.py`, then re-flash the base ESP32.
3. Reprint only the parts named in the "Reprint after" column. If you have not printed them yet, print after step 2.

Numbers still assumed in params.py right now

params.py version: `0.1-stub`.

params.py name	State	Who fixes it
WIPER	assumed, not measured yet	MEASURE_LIST
SPRING	assumed, not measured yet	MEASURE_LIST
LAYOUT	not decided yet (None)	coordinator
DRUM	not decided yet (None)	coordinator
LAUNCH	not decided yet (None)	coordinator
AIM	not decided yet (None)	coordinator
REEL	not decided yet (None)	coordinator
FRAMEWORK	not decided yet (None)	coordinator
SWITCHES	not decided yet (None)	coordinator
TIMELINE	not decided yet (None)	coordinator

15. Sources and appendices

This guide is built by `docs/build/make_guide.py` from the files below. A "NOT YET" row is a section that still shows a placeholder; re-run the script after it lands.

Source	Written by	In the repo
`base/spec/BASE_SPEC.md`	coordinator	yes
`base/pack`	cad-infra (print pack)	NOT YET
`base/out`	cad-infra (assembly pipeline)	yes
`base/guide`	cad-infra (assembly steps)	NOT YET
`kedge/pack_v30/KEDGE_v30_run_sheet.pdf`	dart v30 pack (done)	yes
`firmware/base/WIRING.md`	fw-base	NOT YET
`firmware/base/README.md`	fw-base	NOT YET
`firmware/base/CALIBRATION.md`	fw-base	NOT YET
`firmware/base/BENCH_TEST.md`	fw-base	NOT YET
`firmware/base/PROTOCOL.md`	fw-base	NOT YET
`firmware/dart/WIRING.md`	fw-dart	NOT YET
`firmware/dart/README.md`	fw-dart	NOT YET
`firmware/dart/API.md`	fw-dart	yes
`firmware/dart/BENCH_TEST.md`	fw-dart	NOT YET
`jetson/README.md`	jetson	yes

Appendices attached after this page (when present): the base assembly steps PDF from `base/guide`, then the KEDGE v30 dart run sheet. In the run sheet, follow section 6 of this guide for the new dart pin map.

Other documents in the repo: `kedge/pack_v30/KEDGE_v30_engineering_report.pdf` (dart physics and loads), `base/spec/BASE_SPEC.md` (base design), `firmware/base/PROTOCOL.md` (base serial protocol), `firmware/dart/API.md` (dart HTTP API), `jetson/INTERFACES.md` (Jetson software).

KEDGE dart v30: build day run sheet

Dart: Ø42 x 152 mm, 1.5 mm walls, PLA on the X2D. Kick leg 0 to 201.75°, drill pair 0 to 42.25°. Collision sweep clean (kick in 102 steps to 201.75°, drills in 1° steps to 42.25°, every sub-part, exact meshes). Printed volume 91.91 cm³ (about 114.0 g if solid; less at 40% infill). About 12 hours wall clock with the two plates printing in the background. Rules for the whole day: the power loop plug stays OUT while you solder, move a wire or plug in USB. Never let the two cells touch metal together. One step at a time; each step says what you should see before you move on.

1. What is in the pack

File	What it is
print/plate_A_hull.3mf	Print plate A: hull top, hull bottom, nose, tail cap, both couplers. Already in print orientation.
print/plate_B_legs_tray.3mf	Print plate B: tray, kick leg, both drill pods, three augers. Already in print orientation.
print/fit_coupon.stl	20 minute test print: checks every fit before the long plates (section 4).
print/*.stl, step/*.step	Every printed part on its own, oriented for printing. STEP is the editable CAD (augers are faceted in STEP).
docs/wire_cut_list.csv	All 53 wires, pad to pad, colour, gauge and cut length (also section 9 of this sheet).
firmware/kedge_dart/	Firmware v3: kedge_dart.ino + kedge_logic.h. Keyboard over USB and WiFi, telemetry /status for the rover, current trip, auto-anchor (foot spins while the kick plants it, drills spin while the pods deploy), auto-on-landing. Compiled on ESP32 core 2.0.17 and 3.3.12.
firmware/test/	PC simulation of the control logic (all tests pass).
firmware/reel/reel_tension_limit	Reel sketch for the DevKitC + BTS7960: reel-in speed tapers and stops at a set tether tension (motor current or the load cell).
firmware/jetson/kedge_client.py	Rover / laptop client: status, logging, hold-a-key, auto, terminal driving, FPV overlay.
docs/KEDGE_*_engineering_report.pdf	Flight, launch and landing loads, holding force by soil, reel limit, launcher silhouette, design versions.
launcher/barrel_profile.dxf / barrel_keepout.stl	What the launcher barrel has to clear, with 1 mm clearance.
source/	The parametric generator and the whole check pipeline (run_all.sh). Every number lives in P at the top; KOV overrides make variants.

2. Parts and tools to have on the bench

Item	Qty	Note
XIAO ESP32S3, DRV8833 x3, INA219, BNO085, MP1584 (the 5.04 V one)	1 each	Headers clipped off (flush cutters), INA219 screw terminal cut off. MP1584 #2 stays spare.
OTOTEC 2S BMS stick	1	Caliper it when it arrives: the pocket under the cells is 40.8 x 8 x 2.1 plus 0.5 each way.
18350 cells, charged	2	Charge both to 4.2 V in the XTAR first.
Worm N20 (kick, deploy) + third worm N20 (foot, buy)	3	Buy the foot one single-shaft if listed; otherwise cut its lower stub to 2 mm.
Drill motors (12 x 10 x 24.27, M3 x 56 shaft)	2	
F693ZZ flanged bearings	5	2 per pod + 1 in the kick ear (bag has 10).
M3 nuts / M3 nylocs	2 / 2	Nut against each pod's inner bearing, nyloc in each auger.
M3 x 8 screws	7 + 2	7 countersunk (flat head, DIN 7991) for the radial hull screws, they sit flush with the skin; 2 pan head for the pod retainers (with washers).
M3 x 12 screw, M3 x 10 screw	1 + 1	x12 goes through the ear root into tray disc 1; x10 is the kick hub screw.
M3 washers	3	2 pod retainers + 1 between kick hub and ear bearing.
M3 grub screw	1	Foot auger on the foot N20 shaft flat.
2xAA battery holders (for their contacts)	2	You need 2 springs and 2 flat plates. Launch throws the cells backwards, so springs go at the FRONT of the cell bay and flat plates at the BACK.
470 to 1000 µF capacitor, 10 V or more	1	Across the MP1584 5 V output: rides through contact bounce on landing.
Schottky diode 1N5817 or SS14	1	In wire 18 (5 V to the XIAO): stops back-feeding USB.
Nose ballast 15 g		Steel shot or 3 to 4 M6 nuts set in 5-minute epoxy in the nose cavity. Without it the dart tumbles unless the tether is taut (engineering report, section 3).
JST-XH 3-pin pair, JST-XH 2-pin pair + a 2-pin shorting plug	1 each	Charge socket and power loop in the tail window.

Item	Qty	Note
Silicone wire 22, 24, 26, 28 AWG		Totals per colour in section 9.
Zip ties 2.5 mm, medium threadlocker (or CA), double-sided foam tape, hot glue		
Already on your bench		Caddx Ant camera, Zeus VTX + u.FL antenna, 3S LiPo + XT60 lead #1, XTAR MC2 (charges the 18350s out of the dart), breadboard and jumpers.
Tools		Soldering iron, solder, flux, multimeter, wire strippers, flush cutters, small flat screwdriver (MP1584 trimmer), heat shrink + lighter or heat gun, JST-XH crimper or pre-crimped pigtails, 5-minute epoxy, hex keys, tweezers, luggage scale (pull test).

3. Timeline

Clock	Step	Printer
0:00	4. Fit coupon	coupon (20 min)
0:30	5. Start plate A, flash the firmware	plate A (about 3 h)
1:00	6. Prepare boards, set both MP1584s, cut and label all wires	plate A
2:00	7. Solder the bench harness, power-up test, motor directions	plate A
3:30	Start plate B. Clean plate A parts, test-fit the hull, nose ballast (8a)	plate B (about 2.5 h)
4:00	8b. Battery contacts and BMS wiring off the dart	plate B
6:00	8c. Pods, couplers, augers (plate B done)	
7:00	8d. Populate the tray, battery power-up check	
8:30	8e. Close the hull, couplers, pods	
9:30	8f. Kick leg and foot	
10:15	10. Function test and calibration	
12:00	Done. Charge the cells (section 10, Charging).	

Bambu Studio settings (both plates): PLA, 0.4 nozzle, 0.16 mm layers, 4 wall loops (the 1.5 mm walls come out solid), 40% gyroid infill, top/bottom 5 layers, supports OFF, brim ON (auto) for the augers and couplers, X-Y hole compensation from the coupon (section 4). Plate times above are estimates; Studio shows the real ones.

Part	Print pose	Size on bed mm	Volume cm³
hull_top	split face down (on the bed), saddle up	121 x 50 x 35	17.41
hull_bottom	split face down, belly up	121 x 42 x 21	11.5
nose	tip down on the bed (flat Ø22), spigot up	42 x 42 x 27	8.25
tail_cap	end face down, spigot up (no supports)	42 x 42 x 15	7.13
tray	keel down on the bed; discs stand up, decks bridge 26 mm between discs	109 x 39 x 36	15.98
kick_leg	on its side (pole axis horizontal, hub face down)	119 x 15 x 15	8.7
drill_leg_L	upright, motor pocket open to the top (as installed folded)	46 x 15 x 13	3.37
drill_leg_R	upright, motor pocket open to the top (as installed folded)	46 x 15 x 13	3.37
drill_leg_L_auger	base down, nut slot sideways	17 x 17 x 73	5.5
drill_leg_R_auger	base down, nut slot sideways	17 x 17 x 73	5.5
foot_auger	base down	17 x 17 x 40	3.06
coupler_L	standing on its socket end	7 x 6 x 36	1.24
coupler_R	standing on its socket end	7 x 6 x 27	0.9

4. Fit coupon (0:00, 20 minutes)

Print fit_coupon.stl. It has, left to right: the worm N20 pocket, the drill motor pocket, the F693ZZ seat with its flange recess, the hull boss bore, the N20 D-bore, an M3 pilot, an M3 clearance hole, the pod D-bore, the 3.8 shaft clearance, a coupler peg, and a hull ring with its spigot ring.

Check	Should feel	If not
Worm N20 into the square pocket	Slides in by hand, no wobble	Tight: hole compensation +0.05. Loose: -0.05.
Drill motor into its pocket	Slides in by hand	Same as above
F693ZZ into the seat	Firm thumb press, flange flush in the recess	Too tight: +0.05. Falls out: a drop of CA on assembly day.
Coupler peg into the boss bore	Turns freely, little play	Tight: sand the peg lightly
Coupler peg D-socket onto an N20 shaft	Pushes on firmly, no rotation slop	Loose: a drop of CA on assembly
Coupler peg into the pod D-bore	Slides in, no rotation slop	
M3 screw into the 2.5 pilot	Cuts its own thread, holds	
Spigot ring into the hull ring	Push fit, no rattle	Loose: one layer of tape on the spigots

Directions used below: left = the kick-leg side, seen from the tail with the saddle on top (y plus in the CAD). The left pod is the brighter red one in the viewer.

5. Start plate A and flash the firmware (0:30)

1. Send plate_A_hull.3mf to the X2D.
2. In the Arduino IDE: board XIAO_ESP32S3, Tools > USB CDC On Boot: Enabled (the sketch will not compile otherwise). Library Manager: install Adafruit INA219 and Adafruit BNO08x.

3. Nothing else connected to the XIAO. Plug it in by USB-C, open firmware/kedge_dart/kedge_dart.ino, Upload. Re-select the board if the IDE switches it to the wrong one.
4. Serial Monitor at 115200, 'No line ending'. You should see the KEDGE ready help line (INA219 and BNO085 'not found' is normal now). If the monitor stays blank: press the XIAO reset button once, check the port and USB CDC On Boot.
5. On your phone or laptop, join WiFi KEDGE (password grappstar) and open 192.168.4.1: you get the control page. From now on you can also upload over WiFi: Arduino IDE > Port > the network port called kedge. Unplug USB.

6. Prepare boards and wires (1:00)

1. Clip every header pin flush on the XIAO and the three DRV8833 boards (flush cutters, close to the board). Cut the INA219 screw terminal off. Tin each pad you will use.
2. MP1584s, with the 3S LiPo on XT60 lead #1 and the multimeter on the output, turning the small brass screw with a small screwdriver: the dart one to 5.04 V; the spare one to 8.0 V (this is your bench motor supply in step 7). Disconnect.
3. Cut all wires to the lengths in section 9. Strip 3 mm each end, tin, and put a tape flag with the wire number on each. Always slide heat shrink onto a wire before you solder its second end.
4. DRV8833 check before any 3V3 wire: the driver chip has no logic supply pin. The pin called VIN (or VCC on some red boards) is the MOTOR supply. The 3V3 wires (25 to 27) go only to the SLP / nSLEEP / EEP pad, which wakes the driver. Meter on beep: touch the pad you plan to use and the VIN/VM pin. If it beeps, it is the motor supply: do not put 3V3 on it, or the battery voltage reaches the XIAO.
5. Diode: wire 18 has a Schottky diode (1N5817 or SS14) in it, stripe toward the XIAO, in heat shrink. It stops the 5 V buck back-feeding your Mac when USB is plugged in. Still never plug USB in while the dart is powered.

7. Solder the bench harness and test it (2:00)

Lay the boards on the bench in the order they sit in the dart (picture in section 11): sled boards (MP1584, DRV#3), XIAO + DRV#1, DRV#2 + INA219, BNO085. Solder in this order and check after each group:

1. Power: wires 10 to 13 and 15 to 24, and wire 14 at its MP1584 end only (its BMS end comes in 8b). Fit the capacitor across MP1584 OUT+ / OUT-: long leg to OUT+, the striped leg to OUT-. Snap the u.FL antenna onto the VTX now: never power a VTX without its antenna. Also solder wire 9 at its INA219 VIN+ end only (its socket end is crimped in 8b). Meter on beep: red rail to black rail must NOT beep.
2. 3V3 and ground to the logic: wires 25 to 31.
3. I2C: wires 32 to 35. Motor control: wires 36 to 43. Bridge AIN1 to BIN1 and AIN2 to BIN2 on DRV#2 with a short wire offcut.
4. Motor leads: solder wires 44 to 53 to the DRV outputs only; leave the far ends free.
5. Power-up test with the bench supply (the 8.0 V MP1584 on the LiPo): its OUT+ to the INA219 VIN+ pad, OUT- to the MP1584 IN- pad (main ground). You should see about 4.7 V on the XIAO 5V pad (5.04 minus the diode), the VTX LED on, and on the WiFi page: i reads about 8 V; z finds 0x40 and 0x4a. Disconnect the bench supply.
6. Motor directions, bench supply on again. Twist each motor lead temporarily onto its wire (no solder yet: the kick, drill and foot leads are threaded through holes later). Hold each key half a second and check:

Key	Motor	Must turn so that	If not
1	deploy N20	the output shaft turns the way that swings the pods DOWN (mark the shaft with a pen and compare with the viewer)	swap the pair on wire 44 / 45
2	kick N20	the hub would swing the leg UP over the nose	swap 46 / 47
3	drills	each drill output shaft turns: left motor clockwise seen from behind the motor, right motor anticlockwise (the augers are mirror images, so both then screw in)	one wrong: swap that motor's pair; both wrong: swap 40 / 41
4	foot N20	the foot N20 output shaft turns clockwise, seen from the shaft end (that screws the right-hand foot auger in)	swap 52 / 53

Note the free current on i for each (October bench: worms 20 to 30 mA, drill pair 60 to 70 mA). Mark each motor's lead that goes to the lower wire number. Solder the deploy N20 now (it never goes through a hole). Disconnect the bench supply.

8. Mechanical assembly

8a. Nose ballast (3:30, plate A done)

Stand the nose on its tip. Drop 15 g of steel shot or 3 to 4 M6 nuts into the cavity around the camera tube (not into the camera pocket) and cover them with 5-minute epoxy. Leave it to set while you carry on.

8b. Battery contacts and BMS, off the dart (4:00)

1. Join the two springs with wire 2 (the series link, 22 AWG) and solder wire 3 (BMS BM) to that link. Solder wire 1 to one flat plate and wire 4 to the other.
2. On the bench, solder wires 1, 3 and 4 to the BMS B-, BM and B+ pads, wire 14 to P-, wire 8 to P+, and wires 5 to 7 to B-, BM, B+ for the charge socket. Crimp or solder the JST-XH sockets on wires 5 to 7 (3-pin) and 8 / 9 (2-pin power loop).

8c. Drill pods, couplers and augers (6:00, plate B done)

1. Press one F693ZZ into the inner seat from the motor pocket side, flange toward the motor. Press the other into the nose end from outside, flange on the nose face.
2. Thread an M3 nut onto the drill motor shaft, about 3 mm from the motor face. Feed the shaft through both bearings, drop the motor into its pocket. Threadlocker on the shaft, then hold the nut with tweezers and turn the motor until the nut seats against the inner bearing. Zip-tie the motor in its groove.
3. Push an M3 nyloc into the side slot of the auger (nylon ring toward the point). Augers are handed: the LEFT pod takes drill_leg_L_auger (right-hand flight), the RIGHT pod takes drill_leg_R_auger (left-hand flight). Threadlocker on the shaft end, screw the auger on until its base boss touches the outer bearing.
4. Check: each auger spins freely with the motor, no rub on the pod nose.

8d. Populate the tray (7:00)

1. Glue the joined springs into the long recess on the FRONT cell-bay disc (nose side), one spring in front of each cell position. Glue the wire 1 plate into the BACK disc recess on the right (cell 1 side) and the wire 4 plate on the left. Lay the BMS in the channel under the cell bay.
2. Deploy N20 into the sled cradle (worm box toward the tail, output shaft across), zip tie through the groove under the cradle.
3. Stick the boards on with foam tape: DRV#3 on the keel and MP1584 on the shelf in the sled; BNO085 on deck A; XIAO (USB toward the nose) and DRV#1 on deck B; DRV#2 and INA219 on deck C; VTX on the deck over the cells. Route bundles through the notches in the discs. Antenna lead to the tail.

4. Push both sockets into the tail cap window, antenna through the top slot (wire 9 and wire 14 are already soldered from 7.1 and 8b). Tape over the charge socket now: its pins are live as soon as the cells go in.
5. Cells in: cell 1 (right) button toward the nose, cell 2 (left) button toward the tail. Each pushes in sideways, squeezing its spring, and sits hard against its back plate.
5a. Loop still out. Meter on the charge socket pins (lift the tape corner): pin 1 to 2 reads 3.6 to 4.2 V, pin 2 to 3 reads 3.6 to 4.2 V, pin 1 to 3 reads 7.2 to 8.4 V. Any negative reading: pull the cells out and check their direction.
6. Battery power-up check: check the u.FL antenna is still clicked onto the VTX, then plug the loop in. WiFi page: i reads 7.4 to 8.4 V; z finds 0x40 and 0x4a; the deploy N20 runs on 1. Pull the loop plug.

8e. Close the hull (8:30)

1. Kick N20 into the saddle on the hull top, worm box toward the nose, output shaft through both side holes. Zip tie: down one slot beside the saddle, under the skin, up the other, over the motor. Its leads go down through the hole in the saddle floor.
2. Camera into the nose pocket from the back (lens into the hole), cable out the back. Tray into the bottom half, and seat the nose spigot in the bottom half at the same time: the groove on the spigot sits over the half-rib inside the hull.
3. Thread the leads before the top half goes on: kick leads (wires 46, 47) up through the saddle floor hole; foot leads (52, 53) out through the skin hole beside the saddle; drill leads (48 to 51) out through the two flank holes. Lower the top half on.
4. Radial screws into disc 1: M3 x 8 at left z -3, right z +3 and right z -3, and the M3 x 12 through the ear root at left z +3; four M3 x 8 into the tail spigot (z ±3 both sides). The top half's rib closes the nose groove, so the nose is locked.
5. Solder the kick N20 and the two drill motors to their leads (as marked in 7.6), heat shrink (slide it on before soldering; iron on each joint 3 s at most; shrink with a heat gun on low, no lighter near the PLA).
6. Couplers: slide coupler_L in through the left boss, coupler_R through the right, turning gently until each D-socket drops onto the deploy N20 stub. Pods onto the coupler ends (D-bore), then M3 x 8 + washer into each coupler end. Fold the pods along the flanks.

8f. Kick leg and foot (9:30)

1. Press an F693ZZ into the ear from outside, flange in the recess.
2. Foot N20 into the foot housing (opening on the inboard side), output shaft up through the top hole, zip tie. Foot auger onto the shaft, grub screw onto the flat.
3. Feed the foot leads from the foot pocket through the pole channel and out the small hole near the hub. Solder them to wires 52 and 53 (as marked in 7.6): heat shrink on first, iron 3 s at most per joint, heat gun on low, no lighter near the PLA. Leave the loop round the hub.
4. Push the hub onto the kick N20 stub (D flat), put the washer between hub and ear, M3 x 10 through the ear bearing into the hub. Plug the loop in and send @ until the leg rests folded on the hull.

10. Function test and calibration (10:15)

Use a tray of damp soil or sand. Laptop on the KEDGE WiFi page; hold a key to run, let go to stop. Space bar stops everything.

Key	Should happen	If it goes the wrong way
First, in the air	Hold each key for half a second: directions were fixed in 7.6	If one is still wrong, swap it at the DRV end (open the hull)
2 (hold)	Kick leg swings up over the nose and down; the foot auger tip meets the soil near 180°	Swap wires 46 and 47
2 and 4 (hold both)	Foot auger screws IN while the kick keeps pushing, until the foot housing sits on the soil	Foot backs out: swap wires 52 and 53
1 and 3 (hold both)	Both pods swing down and back while BOTH augers spin and cut in; the dart does not twist	Pods go up: swap 44 and 45. One auger backs out: swap that motor's pair (48/49 or 50/51). Both: swap 40 and 41 at the XIAO
i during each	Note the free current, the current when the foot touches, and the stall current	Put them in kedge_logic.h (kickContactMa, kickStallMa, deployStallMa) and upload over WiFi
b	Roll and pitch of the planted dart	
a	Full auto sequence, same order as above, by itself	Space aborts
A then throw	Arms auto-anchor on landing (the dart anchors 1.5 s after it comes to rest)	Press A again to disarm

Charging: either take the cells out and use the XTAR MC2, or plug a 2S balance charger (JST-XH 3-pin) into the charge socket in the tail. Loop plug OUT while charging.
Recovery: # (drills out), ! (pods up), $ (foot out), @ (kick back). The worms hold every position with the power off.
Note: STOP (space, /stop, or a client hold ending) also aborts a running auto sequence; that is deliberate.
Pin change from the October bench: the drill pair is now on D8/D9 and the foot on D6/D7. D6/D7 are the XIAO's serial pins and flicker at every reset; the slow foot worm does not care, the drills would twitch.

9. Wire cut list (pad to pad)

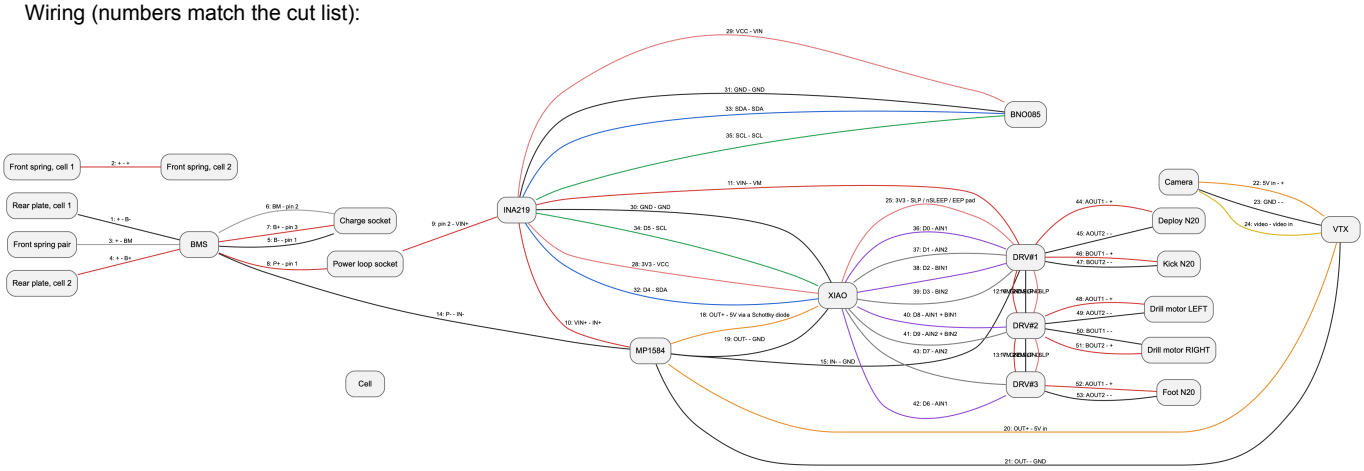
#	From	To	Colour	Gauge	Cut mm	Note
1	Rear plate, cell 1 (-)	BMS B-	black	22 AWG	45	solder to the plate tab before it goes in its recess
2	Front spring, cell 1 (+)	Front spring, cell 2 (-)	red	22 AWG	45	the series link: carries the full battery current; glue both springs in the front recess
3	Front spring pair (cell 1 + / cell 2 -)	BMS BM	white	26 AWG	55	the two springs are joined by a short wire; this is the midpoint sense
4	Rear plate, cell 2 (+)	BMS B+	red	22 AWG	45	
5	BMS B-	Charge socket pin 1 (tail window)	black	26 AWG	80	JST-XH 3-pin, balance charge
6	BMS BM	Charge socket pin 2	white	26 AWG	125	
7	BMS B+	Charge socket pin 3	red	26 AWG	80	
8	BMS P+	Power loop socket pin 1 (tail window)	red	22 AWG	80	JST-XH 2-pin; the shorting plug is the ON switch
9	Power loop socket pin 2	INA219 VIN+	red	22 AWG	95	main + after the loop
10	INA219 VIN+ (same pad)	MP1584 IN+	red	22 AWG	140	buck taps before the shunt, so INA219 reads motor current only
11	INA219 VIN-	DRV#1 VM	red	22 AWG	50	motor rail 6.0-8.4 V
12	DRV#1 VM	DRV#2 VM	red	22 AWG	65	daisy chain
13	DRV#2 VM	DRV#3 VM	red	22 AWG	155	forward into the sled (keel)
14	BMS P-	MP1584 IN-	black	22 AWG	130	main ground
15	MP1584 IN- (same pad)	DRV#1 GND	black	22 AWG	120	
16	DRV#1 GND	DRV#2 GND	black	22 AWG	65	
17	DRV#2 GND	DRV#3 GND	black	22 AWG	155	
18	MP1584 OUT+ (5.04 V)	XIAO 5V via a Schottky diode (stripe toward the XIAO)	orange	24 AWG	160	set and check 5.04 V BEFORE connecting; the diode stops the buck back-feeding USB (1N5817 or SS14, in heat shrink)
19	MP1584 OUT-	XIAO GND	black	24 AWG	160	
20	MP1584 OUT+	VTX 5V in	orange	26 AWG	95	
21	MP1584 OUT-	VTX GND	black	26 AWG	95	
22	VTX 5V in (same pad)	Camera + (red)	orange	28 AWG	105	runs forward along the top through the disc notches
23	VTX GND (same pad)	Camera - (black)	black	28 AWG	105	
24	Camera video (yellow)	VTX video in	yellow	28 AWG	105	twist with the camera ground
25	XIAO 3V3	DRV#1 SLP / nSLEEP / EEP pad (NOT VCC or VIN)	red/white	28 AWG	55	wakes the driver. Check first: if the pad you picked beeps to VIN/VM on the meter, it is the motor supply: do NOT connect 3V3 to it
26	DRV#1 SLP	DRV#2 SLP	red/white	28 AWG	65	
27	DRV#2 SLP	DRV#3 SLP	red/white	28 AWG	155	
28	XIAO 3V3	INA219 VCC	red/white	28 AWG	60	
29	INA219 VCC	BNO085 VIN	red/white	28 AWG	60	
30	XIAO GND	INA219 GND	black	28 AWG	60	
31	INA219 GND	BNO085 GND	black	28 AWG	60	
32	XIAO D4	INA219 SDA	blue	28 AWG	60	I2C SDA
33	INA219 SDA	BNO085 SDA	blue	28 AWG	60	
34	XIAO D5	INA219 SCL	green	28 AWG	60	I2C SCL
35	INA219 SCL	BNO085 SCL	green	28 AWG	60	
36	XIAO D0	DRV#1 AIN1	purple	28 AWG	55	deploy
37	XIAO D1	DRV#1 AIN2	grey	28 AWG	55	deploy
38	XIAO D2	DRV#1 BIN1	purple	28 AWG	55	kick
39	XIAO D3	DRV#1 BIN2	grey	28 AWG	55	kick
40	XIAO D8	DRV#2 AIN1 + BIN1 (bridge the two pads)	purple	28 AWG	45	drills (moved off D6/D7: those twitch at reset)
41	XIAO D9	DRV#2 AIN2 + BIN2 (bridge the two pads)	grey	28 AWG	45	drills
42	XIAO D6	DRV#3 AIN1	purple	28 AWG	145	foot (slow worm, harmless if it twitches at reset)
43	XIAO D7	DRV#3 AIN2	grey	28 AWG	145	foot

#	From	To	Colour	Gauge	Cut mm	Note
44	DRV#1 AOUT1	Deploy N20 + (red)	red	26 AWG	165	motor sits in the sled cradle
45	DRV#1 AOUT2	Deploy N20 - (black)	black	26 AWG	165	
46	DRV#1 BOUT1	Kick N20 + (red)	red	26 AWG	150	out through the hole in the saddle floor
47	DRV#1 BOUT2	Kick N20 - (black)	black	26 AWG	150	
48	DRV#2 AOUT1	Drill motor LEFT + (red)	red	26 AWG	195	out the left flank hole, 30 mm loop for the pod swing
49	DRV#2 AOUT2	Drill motor LEFT - (black)	black	26 AWG	195	
50	DRV#2 BOUT1	Drill motor RIGHT - (black) SWAPPED	black	26 AWG	170	reversed on purpose: the pair counter-rotates
51	DRV#2 BOUT2	Drill motor RIGHT + (red) SWAPPED	red	26 AWG	170	
52	DRV#3 AOUT1	Foot N20 + (red)	red	26 AWG	255	out the skin hole, 60 mm loop round the hub, through the pole channel
53	DRV#3 AOUT2	Foot N20 - (black)	black	26 AWG	255	silicone wire, it flexes every deploy

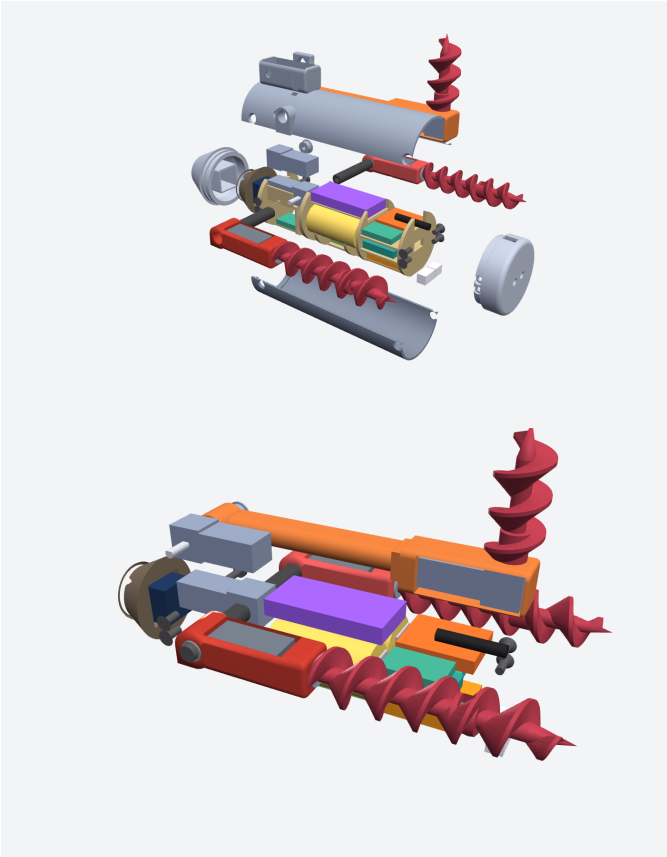
Totals by gauge and colour (mm): 22 AWG black 515, 22 AWG red 675, 24 AWG orange 160, 24 AWG black 160, 26 AWG white 180, 26 AWG black 1110, 26 AWG red 1015, 26 AWG orange 95, 28 AWG orange 105, 28 AWG black 225, 28 AWG yellow 105, 28 AWG red/white 395, 28 AWG blue 120, 28 AWG green 120, 28 AWG purple 300, 28 AWG grey 300

11. Pictures

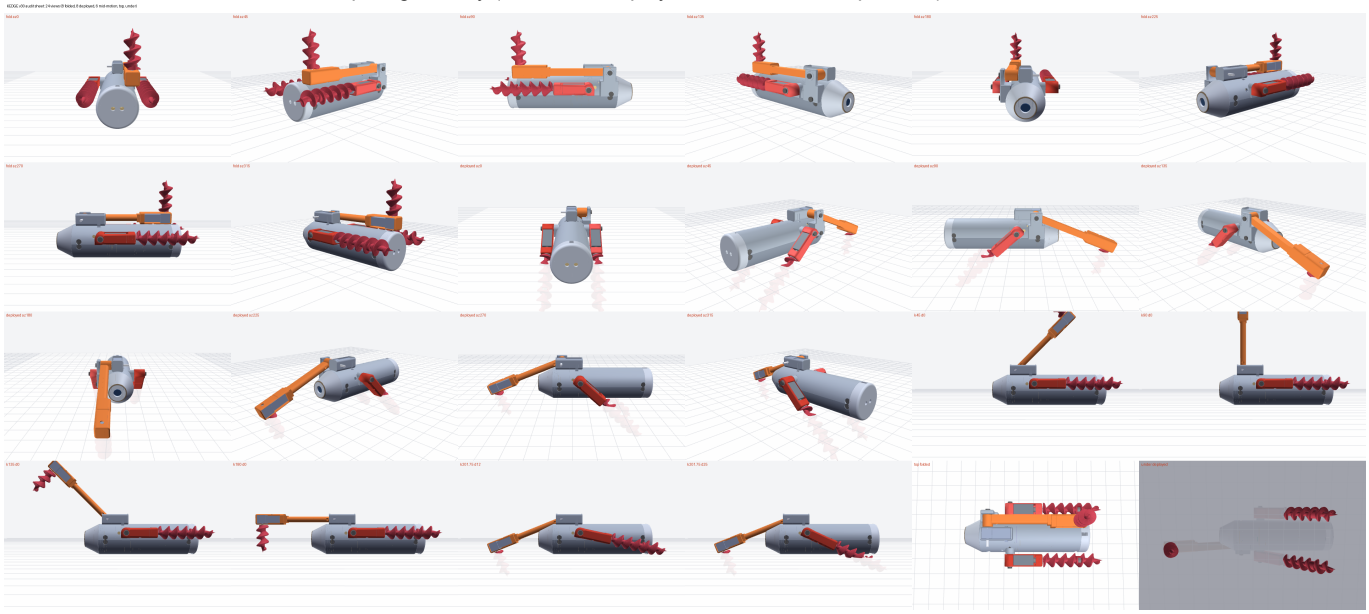
Wiring (numbers match the cut list):



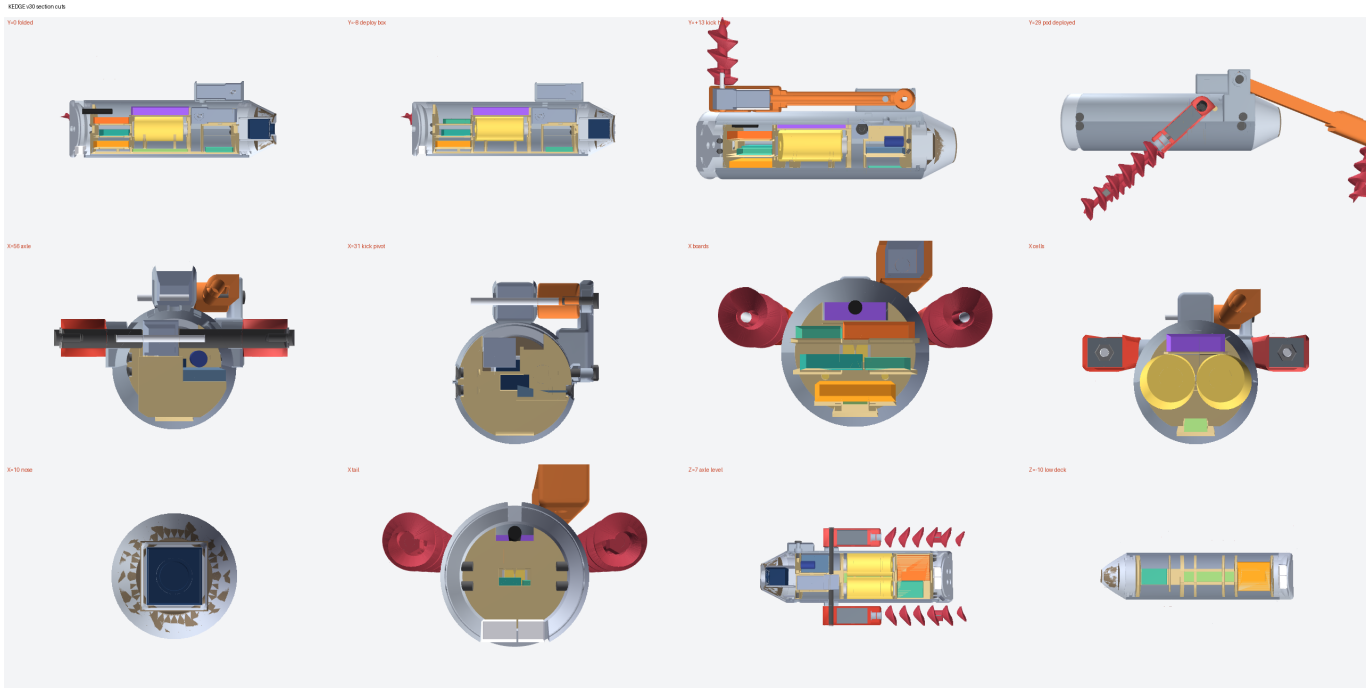
Exploded view and populated tray:



Audit sheet: 24 views of the exact print geometry (8 folded, 8 deployed, 6 mid-motion, top, under).



Section cuts:



12. Design notes

• Pod thrust: the M3 nut against the inner bearing takes the auger's pull into the ground; the auger base against the outer bearing takes push. The gearbox only carries torque.
• The drill pair counter-rotates so the dart does not spin; the two augers are mirror images so both screw in.
• The deploy N20's own shaft stubs drive the pods through two printed Ø7 D couplers that run in the hull bosses.
• Tether load path: tail end wall, tail cap, 4 radial M3 into the hull. Tie the line through the two end holes round the web.
• Ground geometry: kick stops at 201.75° with the foot housing on the ground and the foot auger 35.5 mm in; drills stop at 42.25° with the pods 0.52 mm clear and the augers 44.8 mm in.
• Power: 2S straight to the DRV8833 motor rail through the INA219; one MP1584 at 5.04 V for the XIAO, VTX and camera.
• Below the centreline the hull is a plain Ø42 cylinder for the launcher cradle; the folded pods sit just above it.